esprit
Se former autrement
HONORIS UNITED UNIVERSITIES

DOSSIER DE CONCEPTION TECHNIQUE

EnergyShare

Réseau intelligent de partage d'énergie solaire

Plateforme web distribuée pour les foyers tunisiens

Équipe de réalisation

Fida Sassi

Nour Htira

Chayma Ayari

Chedi Khemiri

INFORMATIONS ACADÉMIQUES

Module : AWD

Matière : Applications Web Distribuées

Encadré par : Ons Ben Salah

Classe : DAS_i11

ANNÉE UNIVERSITAIRE 2026–2027

Table des matières

1 Présentation générale d'EnergyShare **1**
- 1.1 Introduction et contexte tunisien . . . 1
- 1.2 Problématique . . . 1
- 1.3 Objectifs et valeur ajoutée . . . 1
- 1.4 Acteurs et parcours d'utilisation . . . 2
  - 1.4.1 Diagramme de cas d'utilisation global . . . 2
- 1.5 Périmètre fonctionnel et hypothèses . . . 4

2 Architecture et organisation communes **5**
- 2.1 Découpage en cinq microservices . . . 5
- 2.2 Architecture logique globale . . . 5
- 2.3 Bases de données et autonomie . . . 6
- 2.4 Sécurité et infrastructure . . . 6
- 2.5 Scénario global illustratif . . . 7

3 Microservice 1 – User & Household Service **8**
- 3.1 Positionnement et périmètre . . . 8
- 3.2 Fonctionnalités détaillées . . . 8
  - 3.2.1 Gestion du profil utilisateur . . . 8
  - 3.2.2 Gestion des foyers . . . 9
  - 3.2.3 Gestion des membres du foyer . . . 9
  - 3.2.4 Gestion des installations solaires . . . 9
  - 3.2.5 Gestion des préférences énergétiques . . . 9
- 3.3 Modèle de données PostgreSQL . . . 9
  - 3.3.1 Structure des tables . . . 9
  - 3.3.2 Relations et intégrité des données . . . 10
- 3.4 Diagramme de classes du microservice 1 . . . 10
  - 3.4.1 Représentation UML . . . 10
  - 3.4.2 Description des entités . . . 11
- 3.5 Règles métier principales . . . 12
- 3.6 Architecture interne Spring Boot . . . 13
- 3.7 Contrats API REST prévus . . . 14
  - 3.7.1 Profil utilisateur . . . 14
  - 3.7.2 Foyers et membres . . . 14
  - 3.7.3 Installations solaires . . . 14
  - 3.7.4 Préférences énergétiques . . . 14

3.7.5 Codes HTTP et gestion des erreurs . . . 14
3.8 Communication inter-microservices . . . 15
3.8.1 Synthèse des interactions . . . 16
3.8.2 Appels entre services et droits d'accès . . . 16
3.9 Événements RabbitMQ . . . 16
3.10 Résilience et cohérence des données . . . 16
3.10.1 Validation des données . . . 16
3.10.2 Disponibilité du service . . . 16
3.10.3 Cohérence après suppression . . . 17
3.10.4 Sécurité des données . . . 17
3.11 Conclusion . . . 17

4 Microservice 2 – Energy Monitoring Service 18
4.1 Positionnement et périmètre . . . 18
4.2 Fonctionnalités détaillées . . . 18
4.2.1 Suivi de la production solaire . . . 18
4.2.2 Suivi de la consommation énergétique . . . 19
4.2.3 Calcul du bilan et du surplus énergétique . . . 19
4.2.4 Historique et statistiques énergétiques . . . 20
4.2.5 Gestion des alertes énergétiques . . . 20
4.3 Modèle de données PostgreSQL . . . 20
4.3.1 Structure des tables . . . 20
4.3.2 Relations et intégrité des données . . . 21
4.4 Diagramme de classes du microservice 2 . . . 21
4.4.1 Présentation du modèle métier . . . 21
4.4.2 Représentation UML . . . 21
4.4.3 Description des entités . . . 22
4.5 Règles métier principales . . . 23
4.6 Architecture interne Spring Boot . . . 24
4.6.1 Diagramme d'architecture technique . . . 24
4.7 Contrats API REST prévus . . . 25
4.7.1 Codes HTTP et gestion des erreurs . . . 25
4.8 Communication inter-microservices . . . 25
4.8.1 Communication avec Household Service . . . 26
4.8.2 Communication avec Marketplace Service . . . 26
4.8.3 Synthèse des interactions . . . 26
4.9 Événements RabbitMQ . . . 26
4.9.1 Événements envisageables . . . 27
4.10 Résilience et cohérence des données . . . 27
4.10.1 Validation des données . . . 27
4.10.2 Gestion de l'indisponibilité des services . . . 27
4.10.3 Cohérence des bilans . . . 27
4.10.4 Sécurité des données . . . 27
4.11 Conclusion . . . 28

5 Microservice 3 – Energy Marketplace Service **29**
5.1 Positionnement et périmètre . . . 29
5.2 Fonctionnalités détaillées . . . 29
5.2.1 Gestion des offres . . . 29
5.2.2 Gestion des demandes . . . 29
5.2.3 Recherche et matching . . . 29
5.2.4 Gestion des réservations . . . 29
5.3 Modèle de données MongoDB . . . 30
5.3.1 Relations et intégrité . . . 30
5.4 Diagramme de classes du microservice 3 . . . 30
5.4.1 Présentation du modèle de données . . . 30
5.4.2 Représentation UML . . . 30
5.4.3 Description des entités . . . 31
5.4.4 Relations entre les entités . . . 32
5.4.5 Cohérence et intégrité des données . . . 33
5.5 Règles métier principales . . . 33
5.6 Architecture interne Spring Boot . . . 34
5.7 Contrats API REST prévus . . . 34
5.8 Communication inter-microservices . . . 35
5.8.1 Événements prévus . . . 35
5.9 Résilience et cohérence . . . 35

6 Microservice 4 – Energy Transaction Service **36**
6.1 Positionnement et périmètre . . . 36
6.2 Fonctionnalités détaillées . . . 36
6.2.1 Création et validation d'une transaction . . . 36
6.2.2 Paiement simulé et confirmation . . . 37
6.2.3 Clôture et annulation . . . 37
6.2.4 Suivi, historique et justificatif . . . 37
6.3 Modèle de données PostgreSQL . . . 38
6.3.1 Structure des tables . . . 38
6.3.2 Relations et intégrité des données . . . 38
6.4 Diagramme de classes du microservice 4 . . . 39
6.4.1 Représentation UML . . . 39
6.4.2 Description des entités . . . 40
6.4.3 Cycle de vie d'une transaction . . . 41
6.5 Règles métier principales . . . 42
6.6 Architecture interne Spring Boot . . . 43
6.7 Contrats API REST prévus . . . 44
6.7.1 Correspondance entre fonctionnalités et API . . . 44
6.7.2 Endpoints . . . 44
6.7.3 Exemples de requêtes et de réponses . . . 45
6.7.4 Codes HTTP et gestion des erreurs . . . 46
6.7.5 Sécurité des accès . . . 46
6.8 Communication inter-microservices . . . 46
6.8.1 Appel synchrone vers le Marketplace . . . 46

6.8.2 Diagrammes de séquence . . . 47
6.8.3 Synthèse des interactions . . . 49
6.9 Événements RabbitMQ . . . 49
6.9.1 Événements publiés par le Transaction Service . . . 49
6.9.2 Événement consommé . . . 50
6.9.3 Garanties de fiabilité . . . 50
6.10 Résilience et cohérence des données . . . 50
6.10.1 Validation des données . . . 50
6.10.2 Gestion des pannes . . . 51
6.10.3 Concurrence et doubles opérations . . . 51
6.10.4 Cohérence entre services . . . 51
6.10.5 Expiration des transactions . . . 51
6.10.6 Sécurité des données . . . 51
6.11 Conclusion . . . 52

7 Microservice 5 – EnergyShare RAG Assistant 53
7.1 Objectif et périmètre . . . 53
7.2 Fonctionnalités principales . . . 53
7.3 Stack technique . . . 54
7.4 Architecture du pipeline RAG . . . 54
7.5 Préparation de la base de connaissances . . . 56
7.6 Cycle d'une question utilisateur . . . 56
7.7 Modèle relationnel et base vectorielle . . . 56
7.8 API REST prévisionnelles . . . 57
7.9 Sécurité, fiabilité et limites . . . 57
7.10 Évaluation de l'assistant . . . 57

8 Organisation du projet et perspectives 58
8.1 Arborescence du dépôt . . . 58
8.2 Risques principaux . . . 58
8.3 Conclusion . . . 58

Chapitre 1

Présentation générale d'EnergyShare

1.1 Introduction et contexte tunisien

En Tunisie, les périodes estivales de forte chaleur s'accompagnent d'une demande importante en électricité. La pression exercée sur le réseau et les interruptions d'alimentation évoquées dans la présentation du projet ont motivé la recherche d'une utilisation plus rationnelle de l'énergie disponible. Parallèlement, l'énergie solaire offre aux foyers équipés une possibilité d'autoproduction et, à certains moments, de surplus. Ces constats conduisent à envisager une plateforme numérique qui améliore la visibilité des ressources énergétiques et rapproche les foyers producteurs des consommateurs.

EnergyShare est un prototype académique de gestion et de partage *simulé* de surplus solaires entre particuliers. Il ne pilote aucun équipement du réseau électrique, ne livre pas physiquement l'électricité et ne constitue pas un dispositif réglementaire de vente d'énergie. Ses relevés sont saisis manuellement et ses échanges sont représentés par des réservations et transactions logicielles.

1.2 Problématique

Question centrale

Comment permettre aux foyers tunisiens de mieux suivre leur production et leur consommation, de valoriser leurs surplus solaires et de coordonner des échanges énergétiques locaux, à travers une plateforme web sécurisée et intelligible ?

1.3 Objectifs et valeur ajoutée

— **Transparence :** proposer une lecture claire des relevés, bilans, offres et transactions.

— **Valorisation :** rendre visibles les surplus énergétiques déclarés et les besoins des foyers.

— **Accessibilité :** offrir une interface web simple aux producteurs et consommateurs.

— **Assistance :** expliquer l'énergie solaire et l'application grâce à un assistant documentaire RAG.

— **Évolutivité :** séparer les capacités métier en microservices indépendants.

1.4 Acteurs et parcours d'utilisation

Acteur	Fonctions principales
Producteur	Gérer son foyer et son installation, déclarer ses relevés, consulter son surplus théorique, proposer une offre, suivre ses réservations.
Consommateur	Consulter ses relevés, rechercher des offres, créer une demande, réserver une quantité, consulter ses transactions simulées.
Administrateur	Superviser les usages autorisés et assurer, selon les droits définis, l'administration de la base documentaire du RAG.

Un même foyer peut agir comme producteur et consommateur. Les opérations sont soumises à l'identité de l'utilisateur et aux droits d'accès associés à son foyer.

1.4.1 Diagramme de cas d'utilisation global

Le diagramme de cas d'utilisation présenté dans la figure 1.1 illustre les principales interactions entre les acteurs et les fonctionnalités de la plateforme **EnergyShare**.

Il met en évidence trois acteurs principaux : le **producteur**, le **consommateur** et l'**administrateur**.

Les fonctionnalités sont réparties entre les cinq microservices de la plateforme, notamment la gestion des foyers, le suivi énergétique, la marketplace, les transactions simulées et l'assistant intelligent RAG.

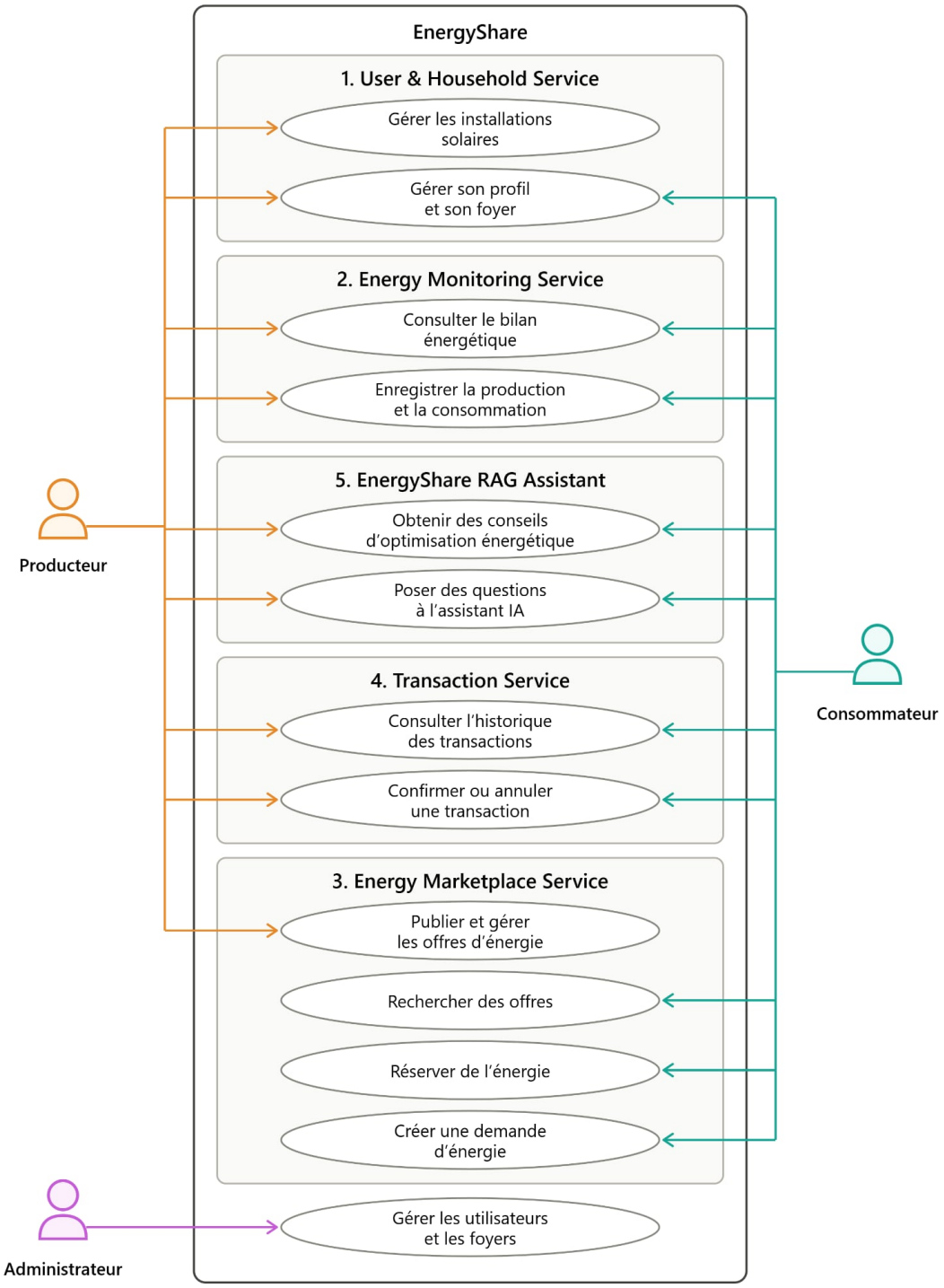


FIGURE 1.1 – Diagramme de cas d'utilisation global de la plateforme EnergyShare.

Ce diagramme permet de visualiser les fonctionnalités accessibles à chaque acteur et de comprendre le périmètre fonctionnel général de la plateforme.

Un même foyer peut jouer les rôles de producteur et de consommateur, selon sa production solaire et ses besoins énergétiques.

1.5 Périmètre fonctionnel et hypothèses

— Les relevés de production et de consommation sont **saisis manuellement** par les foyers ; un relevé quotidien par foyer est envisagé.

— Les prix et les paiements sont **simulés**, à des fins de démonstration.

— La quantité d'énergie affichée comme surplus correspond à un **bilan déclaré ou estimé**, non à une mesure électrique certifiée.

— Les correspondances entre offres et demandes reposent sur des règles de quantité, prix, zone et période.

— Le microservice RAG répond à partir de documents de référence ; il ne remplace pas une prédiction de production ou de consommation.

Chapitre 2

Architecture et organisation communes

2.1 Découpage en cinq microservices

N°	Microservice	Responsabilité principale
1	User & Household Service	Gestion des foyers, profils métier, installations solaires et préférences.
2	Energy Monitoring Service	Saisie manuelle des relevés, calcul des bilans, consultation des surplus et alertes.
3	Energy Marketplace Service	Offres, demandes, recherche, matching par règles et réservations.
4	Energy Transaction Service	Cycle de vie des transactions et paiements simulés.
5	EnergyShare RAG Assistant	Questions-réponses, recherche documentaire augmentée, conseils énergétiques et contexte conversationnel.

2.2 Architecture logique globale

La couche cliente utilise **React et CSS**. Les requêtes de l'interface transitent par **Spring Cloud Gateway**. **Keycloak** centralise l'identité et l'authentification ; **Eureka** et **Config Server** accompagnent les services Spring. Les quatre premiers services métier sont développés en **Spring Boot**, tandis que le cinquième est développé en **Python/Flask**. Le service Flask est joint par une URL configurable via le Gateway et n'exige pas une inscription native dans Eureka.

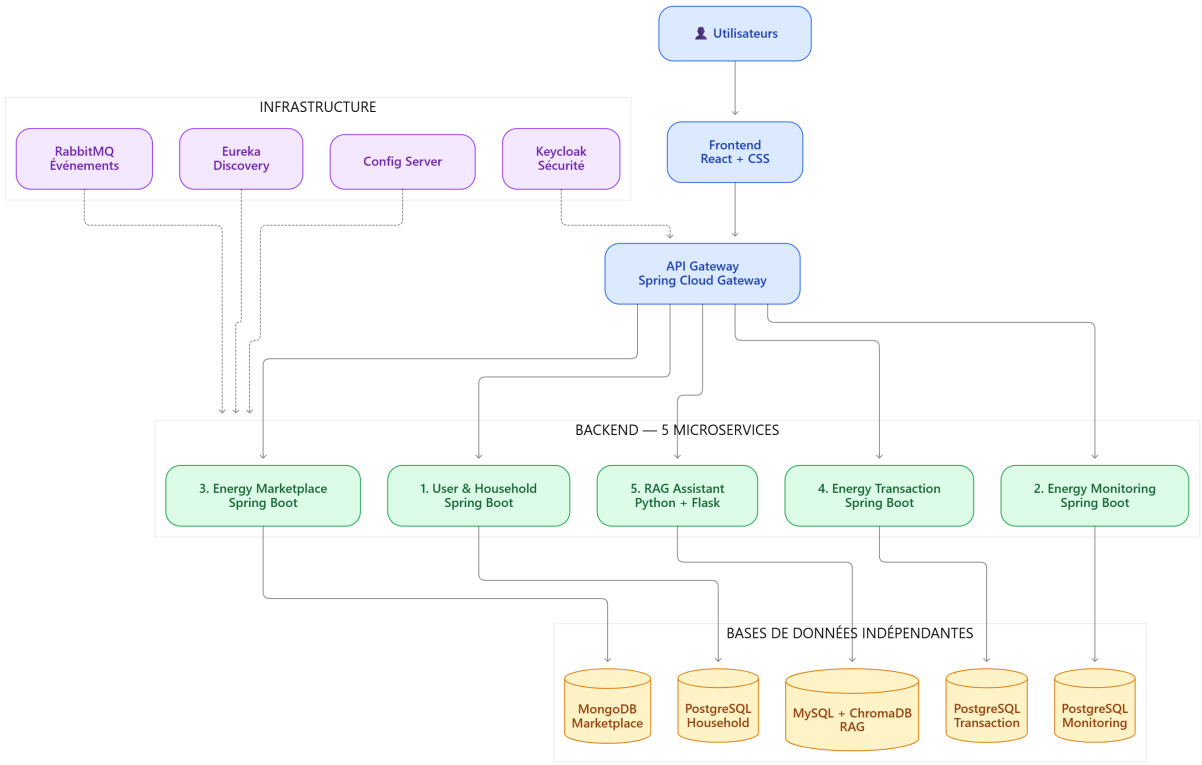


FIGURE 2.1 – Architecture globale de la plateforme EnergyShare.

2.3 Bases de données et autonomie

Chaque service est propriétaire de ses données. Le service Household et le service Monitoring disposent de bases **PostgreSQL** distinctes ; le service Marketplace utilise **MongoDB** ; le service Transaction utilise une autre base **PostgreSQL**. Pour le RAG, **MySQL** conserve les conversations, messages et métadonnées d'utilisation, tandis que **ChromaDB** conserve les vecteurs et passages documentaires. Les comptes et mots de passe sont gérés par **Keycloak**, et non dupliqués comme identité principale dans MySQL.

Les relations entre services sont des **références d'identifiants** et des échanges via API/événements : aucune jointure directe entre bases de données n'est autorisée.

2.4 Sécurité et infrastructure

— **Keycloak :** authentification OAuth2/OpenID Connect ; les API valident les jetons et les autorisations métier.

— **API Gateway :** point d'entrée, routage et contrôles transversaux.

— **Eureka :** découverte dynamique des services Spring Boot.

— **Config Server :** paramètres centralisés pour les services Spring.

— **OpenFeign :** appels REST synchrones, par exemple la vérification d'un foyer et d'un surplus.

— **RabbitMQ :** publication et réception d'événements liés au cycle des réservations et transactions.

— **Docker Compose :** déploiement local reproductible des applications et composants techniques.

2.5 Scénario global illustratif

1. Un foyer s'inscrit et renseigne son installation photovoltaïque dans Household.
2. Il saisit ses données quotidiennes dans Monitoring : par exemple, 20 kWh produits et 12 kWh consommés, soit un bilan net de +8 kWh.
3. Le producteur publie sur Marketplace une offre qui respecte les contrôles de disponibilité prévus.
4. Un consommateur recherche puis réserve une partie de l'offre.
5. Transaction traite la réservation et notifie son état final ; Marketplace actualise l'état de la réservation.
6. Les utilisateurs peuvent consulter le RAG pour obtenir des explications sur la plateforme ou les bonnes pratiques énergétiques.

Chapitre 3

Microservice 1 – User & Household Service

3.1 Positionnement et périmètre

Le **User & Household Service** constitue le microservice de référence pour l'identité métier des utilisateurs et pour la description des foyers inscrits sur la plateforme **EnergyShare**.

Il gère les profils des utilisateurs, les foyers, les membres rattachés à chaque foyer, les installations solaires déclarées et les préférences énergétiques. Il est le **propriétaire** des informations relatives aux foyers : les autres microservices n'en conservent que l'identifiant.

L'authentification (comptes, mots de passe, jetons) est déléguée à **Keycloak**. Le microservice ne stocke aucun mot de passe : il enregistre uniquement un profil métier rattaché à l'identifiant du compte Keycloak, créé après l'inscription de l'utilisateur.

Le périmètre de ce microservice est distinct de celui des autres services :

— **Energy Monitoring Service** : saisie des relevés, calcul des bilans et des surplus.

— **Energy Marketplace Service** : offres, demandes, recherche et réservations.

— **Energy Transaction Service** : transactions et paiements simulés.

— **EnergyShare RAG Assistant** : assistance conversationnelle et recherche documentaire.

Le User & Household Service ne saisit aucun relevé, ne publie aucune offre et n'exécute aucune transaction. Il fournit aux autres services les informations qui permettent de **vérifier l'existence, l'état et les droits d'un foyer**.

3.2 Fonctionnalités détaillées

3.2.1 Gestion du profil utilisateur

Après son inscription auprès de Keycloak, l'utilisateur complète son profil EnergyShare : nom, prénom, coordonnées et informations de contact. Le profil est créé une seule fois et rattaché à l'identifiant du compte Keycloak.

L'utilisateur peut ensuite consulter et modifier son propre profil. Ces opérations s'appuient sur la ressource `/users/me` : l'identité est déduite du jeton d'authentification et non d'un identifiant fourni dans l'URL, ce qui empêche la consultation du profil d'un autre utilisateur.

3.2.2 Gestion des foyers

Un foyer représente une unité de consommation et, le cas échéant, de production d'énergie. Il possède un nom, une adresse, une zone géographique et un statut. Le service permet de créer, consulter, modifier et supprimer un foyer.

Un même foyer peut agir comme producteur et comme consommateur, selon la présence d'une installation solaire et ses besoins énergétiques.

3.2.3 Gestion des membres du foyer

Plusieurs utilisateurs peuvent être rattachés à un même foyer. Chaque membre possède un rôle au sein du foyer (par exemple propriétaire ou membre simple). Le service permet de lister les membres, d'en ajouter et d'en retirer, sous réserve des droits de l'utilisateur qui effectue l'opération.

3.2.4 Gestion des installations solaires

Le producteur déclare une ou plusieurs installations photovoltaïques rattachées à son foyer : puissance installée, nombre de panneaux, date de mise en service et statut. Ces informations servent de référence aux autres services, notamment pour contextualiser les relevés de production du Monitoring.

3.2.5 Gestion des préférences énergétiques

Chaque foyer dispose d'un ensemble de préférences énergétiques (rôle préféré, prix minimal ou maximal acceptable, seuil de surplus à partager, activation des alertes). La ressource est unique par foyer : elle est créée ou remplacée en une seule opération.

Remarque :
Les déclarations du foyer et de ses installations sont **saisies par les utilisateurs**. Elles ne sont pas certifiées par un organisme tiers : EnergyShare reste un prototype académique de partage simulé.

3.3 Modèle de données PostgreSQL

Le microservice utilise **PostgreSQL** comme système de gestion de base de données relationnelle. La persistance est assurée par **Spring Data JPA**.

Le modèle comprend cinq entités principales :

— **UserProfile**

— **Household**

— **HouseholdMember**

— **SolarInstallation**

— **EnergyPreference**

3.3.1 Structure des tables

Table	Champs principaux	Rôle
`user_profiles`	`id`, `keycloakId`, `firstName`, `lastName`, `email`, `phone`, `createdAt`	Profil métier rattaché au compte Keycloak.
`households`	`id`, `name`, `address`, `zone`, `status`, `createdAt`	Description et état du foyer.
`household_members`	`id`, `householdId`, `userId`, `memberRole`, `joinedAt`	Association entre utilisateurs et foyers.
`solar_installations`	`id`, `householdId`, `capacityKwc`, `panelCount`, `commissioningDate`, `status`	Installations photovoltaïques déclarées.
`energy_preferences`	`id`, `householdId`, `preferredRole`, `minPricePerKwh`, `maxPricePerKwh`, `minSurplusKwh`, `alertsEnabled`	Préférences énergétiques du foyer.

3.3.2 Relations et intégrité des données

Les relations internes au microservice sont des clés étrangères PostgreSQL, car toutes les tables appartiennent à la même base :

— un utilisateur peut appartenir à plusieurs foyers et un foyer peut compter plusieurs utilisateurs, via `household_members` ;

— un foyer peut posséder plusieurs installations solaires ;

— un foyer possède au plus un ensemble de préférences (unicité sur `householdId`).

L'identifiant d'un foyer est ensuite repris par les autres microservices (`householdId` dans Monitoring, `sellerHouseholdId` dans Marketplace) sans clé étrangère interservices, conformément au principe *Database per Service*.

3.4 Diagramme de classes du microservice 1

3.4.1 Représentation UML

Le diagramme de classes du **User & Household Service** représente les entités de gestion des utilisateurs, des foyers, des installations et des préférences, ainsi que leurs relations.

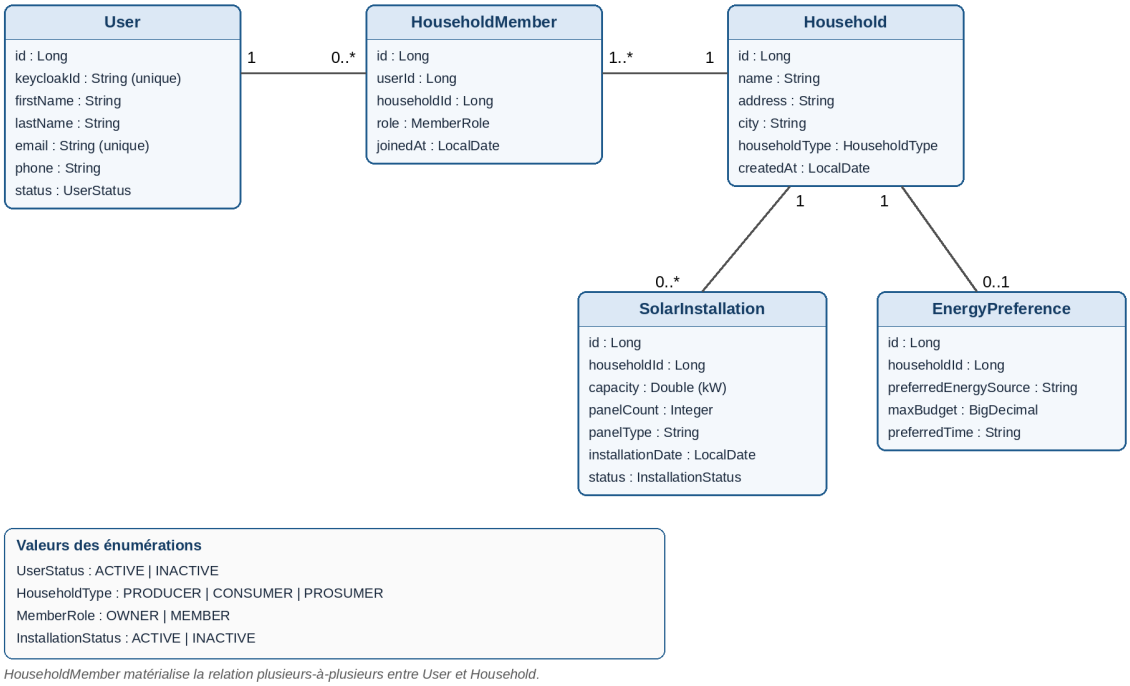


FIGURE 3.1 – Diagramme de classes du User & Household Service.

3.4.2 Description des entités

Entité UserProfile

L'entité **UserProfile** représente le profil métier d'un utilisateur d'EnergyShare.

— **id** : identifiant unique du profil.

— **keycloakId** : identifiant du compte Keycloak associé (unique).

— **firstName**, **lastName** : nom et prénom.

— **email**, **phone** : coordonnées de contact.

— **createdAt** : date de création du profil.

Entité Household

L'entité **Household** représente un foyer.

— **id** : identifiant unique du foyer, utilisé par les autres microservices.

— **name** : nom du foyer.

— **address** et **zone** : localisation du foyer.

— **status** : état du foyer, par exemple ACTIVE ou INACTIVE.

— **createdAt** : date de création.

Entité HouseholdMember

L'entité **HouseholdMember** associe un utilisateur à un foyer.

— **id** : identifiant de l'association.

— **householdId** et **userId** : foyer et utilisateur concernés.

— **memberRole** : rôle dans le foyer, par exemple OWNER ou MEMBER.

— **joinedAt** : date de rattachement.

Entité SolarInstallation

L'entité **SolarInstallation** représente une installation photovoltaïque déclarée par un foyer.

— **id** : identifiant de l'installation.

— **householdId** : foyer propriétaire.

— **capacityKwc** : puissance installée en kWc.

— **panelCount** : nombre de panneaux.

— **commissioningDate** : date de mise en service.

— **status** : état de l'installation, par exemple ACTIVE ou INACTIVE.

Entité EnergyPreference

L'entité **EnergyPreference** regroupe les préférences énergétiques d'un foyer.

— **householdId** : foyer concerné (unique).

— **preferredRole** : PRODUCER, CONSUMER ou BOTH.

— **minPricePerKwh**, **maxPricePerKwh** : bornes de prix souhaitées (prix simulés).

— **minSurplusKwh** : seuil de surplus à partir duquel le foyer souhaite partager.

— **alertsEnabled** : activation des alertes.

Exemple de foyer producteur

Un foyer « Famille Ben Ali » à Tunis déclare une installation de **3 kWc** composée de **8 panneaux**, puis enregistre ses préférences : rôle `PRODUCER`, prix minimal de **0,20 DT/kWh**.

Ces valeurs constituent des propositions de conception à harmoniser avec les entités et énumérations Java du projet.

3.5 Règles métier principales

1. Le profil d'un utilisateur est créé une seule fois, après son inscription, et rattaché à un compte Keycloak.
2. Un utilisateur ne peut consulter et modifier que son propre profil.
3. Tout foyer doit posséder au moins un membre propriétaire.
4. Seuls les membres d'un foyer peuvent le consulter ; seul un propriétaire peut le modifier, le supprimer ou gérer ses membres.
5. Un utilisateur ne peut pas être ajouté deux fois au même foyer.
6. Le dernier propriétaire d'un foyer ne peut pas être retiré.

7. Une installation solaire est toujours rattachée à un foyer existant, et sa puissance doit être strictement positive.
8. Un foyer possède au plus un ensemble de préférences ; l'opération de mise à jour remplace l'existant.
9. La suppression d'un foyer est refusée tant que des éléments dépendants l'interdisent (cas de conflit).
10. Aucune donnée d'un foyer n'est accessible à un utilisateur qui n'en est pas membre.

3.6 Architecture interne Spring Boot

Le **User & Household Service** repose sur une architecture en couches développée avec **Spring Boot**.

— **REST Controllers** : réception des requêtes HTTP, validation et codes de réponse.

— **Service Layer** : règles métier, contrôle des droits d'accès au foyer.

— **Spring Data JPA Repositories** : accès aux données relationnelles.

— **PostgreSQL** : base de données dédiée.

— **Spring Security (OAuth2 Resource Server)** : validation des jetons émis par Keycloak.

Le service est enregistré dans **Eureka**, récupère sa configuration auprès du **Config Server** et est exposé au frontend React à travers l'**API Gateway**. Contrairement aux autres services métier, il est principalement **appelé** par ses voisins : il ne dépend d'aucun d'eux pour accomplir ses fonctionnalités principales.

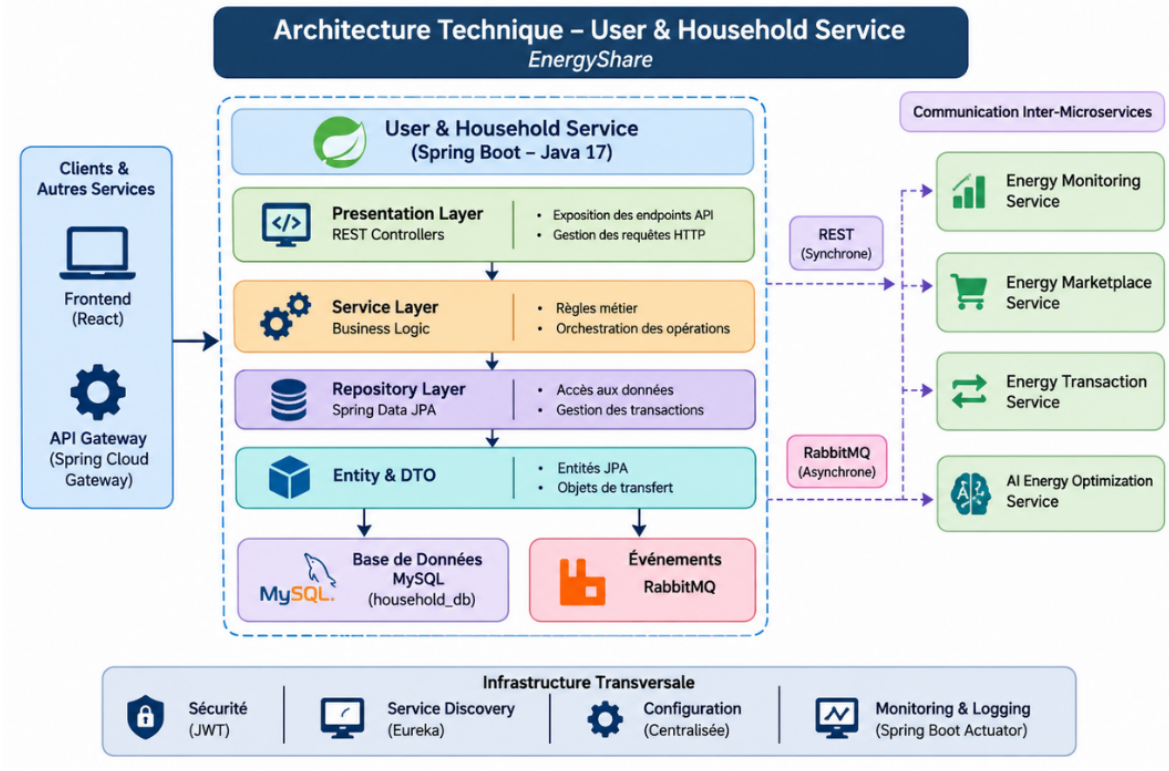


FIGURE 3.2 – Architecture technique du microservice 1 — User & Household Service.

3.7 Contrats API REST prévus

Le microservice expose des API REST regroupées en quatre ensembles fonctionnels. Elles sont accessibles depuis l'interface React via l'API Gateway.

3.7.1 Profil utilisateur

Méthode	URI	Description	Réponses
POST	`/users`	Créer le profil de l'utilisateur après son inscription	201, 400, 409
GET	`/users/me`	Consulter son profil	200, 404
PATCH	`/users/me`	Modifier son profil	200, 400

3.7.2 Foyers et membres

Méthode	URI	Description	Réponses
POST	`/households`	Créer un foyer	201, 400
GET	`/households/{id}`	Consulter un foyer	200, 403, 404
PATCH	`/households/{id}`	Modifier un foyer	200, 400, 403, 404
DELETE	`/households/{id}`	Supprimer un foyer	204, 403, 404, 409
GET	`/households/{id}/members`	Lister les membres d'un foyer	200, 403, 404
POST	`/households/{id}/members`	Ajouter un membre au foyer	201, 400, 403, 404, 409
DELETE	`/households/{id}/members/{memberId}`	Retirer un membre du foyer	204, 403, 404, 409

3.7.3 Installations solaires

Méthode	URI	Description	Réponses
POST	`/households/{id}/installations`	Déclarer une installation solaire	201, 400, 403, 404, 409
GET	`/households/{id}/installations`	Lister les installations d'un foyer	200, 403, 404
PATCH	`/installations/{id}`	Modifier une installation	200, 400, 403, 404
DELETE	`/installations/{id}`	Supprimer une installation	204, 403, 404

3.7.4 Préférences énergétiques

Méthode	URI	Description	Réponses
GET	`/households/{id}/preferences`	Consulter les préférences du foyer	200, 403, 404
PUT	`/households/{id}/preferences`	Créer ou remplacer les préférences	200, 400, 403, 404

3.7.5 Codes HTTP et gestion des erreurs

— `200 OK` : consultation ou modification réussie.

— `201 Created` : création réussie.

— `204 No Content` : suppression réussie.

— `400 Bad Request` : données invalides.

— `403 Forbidden` : l'utilisateur n'est pas autorisé à accéder au foyer.

— `404 Not Found` : ressource introuvable.

— `409 Conflict` : conflit métier (profil ou membre déjà existant, suppression impossible).

Le code `401 Unauthorized` (utilisateur non authentifié) est renvoyé par la couche de sécurité pour l'ensemble des endpoints. Les endpoints présentés constituent des **contrats de conception** à implémenter et à tester dans Spring Boot.

3.8 Communication inter-microservices

Le User & Household Service est le service **appelé** par ses voisins. Les échanges reposent sur des appels REST synchrones, car les services appelants ont besoin d'une réponse immédiate pour valider leur opération.

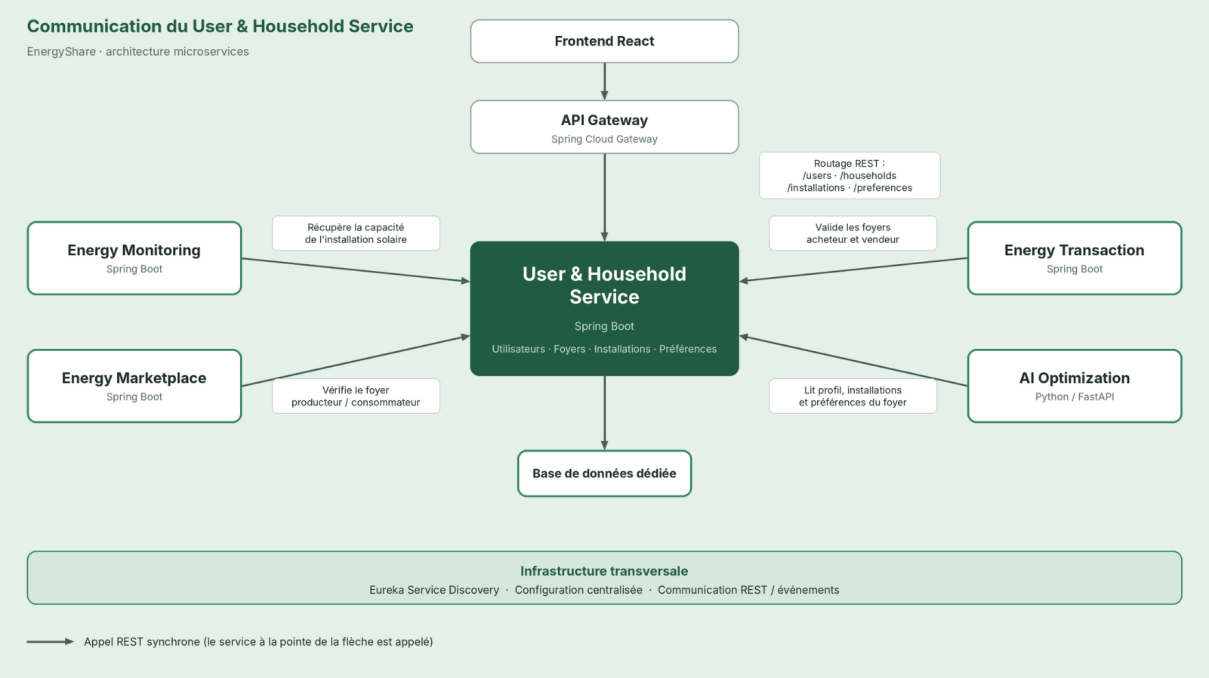


FIGURE 3.3 – Communication du User & Household Service avec les autres microservices.

3.8.1 Synthèse des interactions

Service	Mode	Rôle de la communication
Energy Monitoring Service	OpenFeign / REST	Vérification de l'existence et de l'état d'un foyer avant l'enregistrement d'un relevé.
Energy Marketplace Service	OpenFeign / REST	Vérification de l'existence, du rôle et de l'état du foyer vendeur avant la publication d'une offre.
Energy Transaction Service	À préciser	Aucun appel direct n'est imposé par la conception actuelle ; une vérification des foyers acheteur et vendeur reste envisageable.
RAG Assistant	Aucun appel direct requis	Le RAG s'appuie sur sa base documentaire et sur l'identifiant Keycloak de l'utilisateur.

3.8.2 Appels entre services et droits d'accès

Les API d'un foyer imposent normalement que l'appelant en soit membre. Pour les appels de service à service, un mécanisme d'authentification technique doit être défini, par exemple un jeton de type *client credentials* émis par Keycloak ou un endpoint interne dédié à la vérification d'un foyer. Cette décision reste à harmoniser avec les équipes des microservices appelants.

3.9 Événements RabbitMQ

Dans la conception actuelle, le User & Household Service ne dépend pas de **RabbitMQ** pour ses fonctionnalités principales. Des événements pourraient toutefois être publiés pour informer les autres services d'un changement d'état :

— `HouseholdDeleted` : un foyer a été supprimé.

— `HouseholdStatusChanged` : un foyer est devenu actif ou inactif.

— `InstallationUpdated` : une installation solaire a été créée, modifiée ou supprimée.

Ces événements sont des évolutions possibles et non des dépendances obligatoires. Leurs contrats devront être définis avec les services consommateurs avant toute implémentation.

3.10 Résilience et cohérence des données

3.10.1 Validation des données

Les données reçues sont validées avant stockage : champs obligatoires, formats, puissance installée strictement positive. Les unicités (compte Keycloak, appartenance d'un utilisateur à un foyer, préférences par foyer) sont garanties par des contraintes PostgreSQL.

3.10.2 Disponibilité du service

Comme les services Monitoring et Marketplace refusent une opération lorsqu'un foyer ne peut pas être vérifié, la disponibilité du User & Household Service est importante. Il doit répondre rapidement, rester sans état et pouvoir être déployé en plusieurs instances derrière Eureka.

3.10.3 Cohérence après suppression

Un foyer est référencé par d'autres services (relevés, offres). La suppression doit donc être maîtrisée : refus en cas de conflit, ou désactivation du foyer plutôt que suppression physique, avec éventuelle notification par événement.

3.10.4 Sécurité des données

Les endpoints sont protégés par l'authentification **Keycloak**. L'identité de l'utilisateur est lue dans le jeton, et son appartenance au foyer est vérifiée à chaque requête : connaître l'identifiant d'un foyer ne suffit pas à y accéder.

3.11 Conclusion

Le microservice **User & Household Service** constitue le socle d'identité métier de la plateforme EnergyShare. Il centralise les profils, les foyers, leurs membres, leurs installations solaires et leurs préférences, et fournit aux autres microservices les informations qui permettent de vérifier un foyer avant d'enregistrer un relevé ou de publier une offre.

Grâce à **Spring Boot**, **Spring Data JPA**, **PostgreSQL** et **Keycloak**, il propose une architecture modulaire, sécurisée et indépendante, qui sépare clairement la gestion des foyers des fonctions de suivi, de marché et de transaction.

Chapitre 4

Microservice 2 – Energy Monitoring Service

4.1 Positionnement et périmètre

Le **Energy Monitoring Service** constitue le microservice responsable du suivi énergétique des foyers inscrits sur la plateforme **EnergyShare**.

Son objectif principal est de permettre aux utilisateurs d'enregistrer, de consulter et d'analyser leurs données de production et de consommation électrique.

Il assure également le calcul des bilans énergétiques, l'estimation des surplus disponibles et la génération d'alertes liées aux données énergétiques.

Dans le cadre de notre projet, les relevés sont **saisis manuellement par les utilisateurs**. Le microservice ne communique donc pas directement avec des compteurs intelligents ou des équipements physiques de mesure.

Il permet ainsi de représenter et d'analyser la situation énergétique d'un foyer à partir des informations déclarées.

Le périmètre de ce microservice est distinct de celui des autres services :

— **User & Household Service** : gestion des utilisateurs, des foyers et des installations solaires.

— **Energy Marketplace Service** : publication des offres, gestion des demandes et réservation des surplus énergétiques.

— **Energy Transaction Service** : gestion des transactions et des paiements simulés.

— **EnergyShare RAG Assistant** : assistance conversationnelle et recherche documentaire intelligente.

Le microservice Monitoring ne publie pas d'offres, ne crée pas de réservations et n'effectue aucune transaction financière.

Il fournit principalement les informations énergétiques nécessaires au suivi des foyers et à la consultation du surplus déclaré.

4.2 Fonctionnalités détaillées

4.2.1 Suivi de la production solaire

Le service permet aux foyers producteurs d'enregistrer quotidiennement la quantité d'énergie électrique générée par leurs installations photovoltaïques.

Chaque relevé contient notamment :

— l'identifiant du foyer concerné ;

— la date du relevé ;

— la quantité d'énergie produite en kWh ;

— la date de création de l'enregistrement.

Ces données permettent de suivre l'évolution de la production solaire sur différentes périodes.

4.2.2 Suivi de la consommation énergétique

Le microservice permet également d'enregistrer la quantité d'énergie électrique consommée par chaque foyer.

Les informations sont saisies manuellement dans l'interface web EnergyShare.

L'utilisateur peut ensuite consulter ses relevés historiques afin de mieux comprendre l'évolution de sa consommation.

Ce suivi contribue à identifier les périodes de forte demande énergétique et à sensibiliser les utilisateurs aux bonnes pratiques de consommation.

4.2.3 Calcul du bilan et du surplus énergétique

L'une des principales responsabilités du **Energy Monitoring Service** est de calculer le bilan énergétique d'un foyer sur une période donnée.

Le calcul repose sur la comparaison entre l'énergie produite et l'énergie consommée.

$$B = P - C$$

Avec :

— B : bilan énergétique net (kWh) ;

— P : production totale (kWh) ;

— C : consommation totale (kWh).

Le surplus énergétique théorique est calculé par :

$$S = \max(0, P - C)$$

Lorsque la production est supérieure à la consommation, le foyer présente un surplus positif.

Dans le cas contraire, le surplus est nul et le foyer présente un déficit énergétique net.

Remarque :
Le surplus calculé représente une estimation obtenue à partir des valeurs déclarées.
Il ne garantit pas qu'une quantité d'électricité est physiquement disponible pour un transfert à un instant donné.
Le partage énergétique dans EnergyShare reste une simulation logicielle.

4.2.4 Historique et statistiques énergétiques

Le microservice conserve l'historique des relevés enregistrés par les utilisateurs.

Il permet de consulter les statistiques de production et de consommation selon différentes périodes :

— **Journalière** : consultation des relevés quotidiens.

— **Hebdomadaire** : analyse des données sur une semaine.

— **Mensuelle** : analyse des variations sur un mois.

Ces résultats peuvent être affichés sous forme de tableaux ou de graphiques dans l'interface **React**.

Ils permettent de visualiser les tendances énergétiques et de comparer les périodes.

4.2.5 Gestion des alertes énergétiques

Le service prévoit également un mécanisme d'alertes énergétiques.

Ces alertes peuvent informer les utilisateurs de situations particulières, notamment :

— une consommation supérieure à un seuil défini ;

— une production solaire insuffisante ;

— un déficit énergétique important ;

— la disponibilité d'un surplus énergétique théorique.

Les alertes sont enregistrées et associées au foyer concerné.

Elles possèdent un type, un message, un niveau de gravité et un statut permettant de suivre leur traitement.

4.3 Modèle de données PostgreSQL

Le microservice utilise **PostgreSQL** comme système de gestion de base de données relationnelle.

La persistance est assurée par **Spring Data JPA**, qui permet de manipuler les entités métier à travers des repositories.

Le modèle comprend trois entités principales :

— **EnergyReading**

— **EnergyBalance**

— **EnergyAlert**

4.3.1 Structure des tables

Table	Champs principaux	Rôle
`energy_readings`	`id`, `householdId`, `readingDate`, `productionKwh`, `consumptionKwh`, `createdAt`	Stockage des relevés journaliers de production et de consommation.

Table	Champs principaux	Rôle
`energy_balances`	`id`, `householdId`, `periodStart`, `periodEnd`, `totalProductionKwh`, `totalConsumptionKwh`, `surplusKwh`	Conservation des bilans énergétiques calculés sur une période donnée.
`energy_alerts`	`id`, `householdId`, `alertType`, `message`, `severity`, `createdAt`, `status`	Gestion des alertes énergétiques associées aux foyers.

4.3.2 Relations et intégrité des données

Les trois entités sont associées à un foyer grâce à l'attribut `householdId`.

Cet identifiant fait référence à un foyer géré par le microservice **User & Household Service**.

Il ne constitue pas une clé étrangère vers une autre base PostgreSQL.

En effet, chaque microservice possède sa propre base de données, conformément au principe *Database per Service*.

Un foyer peut posséder :

— plusieurs relevés énergétiques ;

— plusieurs bilans correspondant à différentes périodes ;

— plusieurs alertes énergétiques.

Les liens entre les informations sont établis logiquement grâce à l'identifiant du foyer.

4.4 Diagramme de classes du microservice 2

4.4.1 Présentation du modèle métier

Le diagramme de classes du **Energy Monitoring Service** représente les principales entités chargées du suivi énergétique.

Il met en évidence la structure des données nécessaires à l'enregistrement des relevés, au calcul des bilans et à la gestion des alertes.

4.4.2 Représentation UML

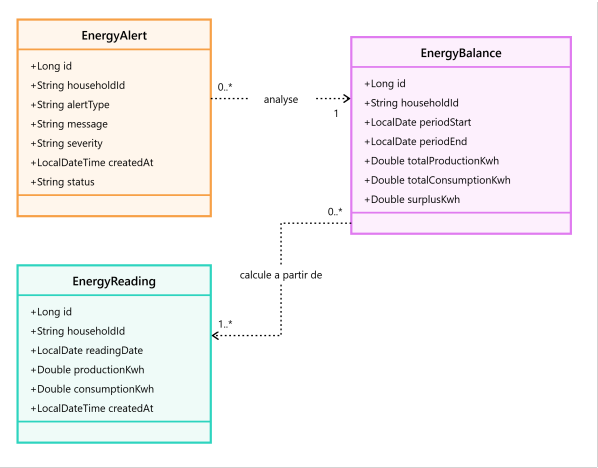


FIGURE 4.1 – Diagramme de classe monitoring service

Diagramme de classes à insérer :
Ajouter le fichier `images/microservice2_classes.png` dans le projet Overleaf.

4.4.3 Description des entités

Entité EnergyReading

L'entité **EnergyReading** représente un relevé énergétique déclaré par un foyer.
Elle permet d'enregistrer les quantités d'énergie produites et consommées à une date donnée.
Ses attributs principaux sont :

— **id** : identifiant unique du relevé, de type Long.

— **householdId** : identifiant du foyer concerné, de type String.

— **readingDate** : date du relevé énergétique, de type LocalDate.

— **productionKwh** : quantité d'énergie solaire produite, de type Double.

— **consumptionKwh** : quantité d'énergie consommée, de type Double.

— **createdAt** : date et heure de création du relevé, de type LocalDateTime.

Chaque relevé permet de comparer la production et la consommation pour une journée donnée.
Dans la conception retenue, un foyer possède au maximum un relevé par date.
Cette contrainte doit être appliquée au niveau de la base de données à travers une unicité sur le couple `householdId` et `readingDate`.

Entité EnergyBalance

L'entité **EnergyBalance** représente le bilan énergétique calculé pour un foyer sur une période donnée.
Elle regroupe les résultats obtenus à partir des relevés enregistrés.
Ses attributs principaux sont :

— **id** : identifiant unique du bilan, de type Long.

— **householdId** : identifiant du foyer concerné.

— **periodStart** : date de début de la période.

— **periodEnd** : date de fin de la période.

— **totalProductionKwh** : quantité totale d'énergie produite.

— **totalConsumptionKwh** : quantité totale d'énergie consommée.

— **surplusKwh** : surplus énergétique théorique calculé pour la période.

Le bilan est calculé en additionnant les valeurs des relevés compris dans la période sélectionnée.
Le surplus est obtenu grâce à la différence positive entre la production totale et la consommation totale.

L'entité peut être utilisée pour conserver les bilans calculés et éviter de répéter certaines opérations d'agrégation.

Les bilans stockés doivent être actualisés ou recalculés lorsqu'un relevé associé est modifié ou supprimé.

Entité EnergyAlert

L'entité **EnergyAlert** représente une notification associée à une situation énergétique particulière.

Elle permet d'informer les utilisateurs lorsqu'une condition métier prédéfinie est détectée.

Ses attributs principaux sont :

— **id** : identifiant unique de l'alerte.

— **householdId** : identifiant du foyer concerné.

— **alertType** : type de l'alerte énergétique.

— **message** : description de la situation détectée.

— **severity** : niveau de gravité de l'alerte.

— **createdAt** : date et heure de génération.

— **status** : état actuel de l'alerte.

Les niveaux de gravité peuvent être représentés par :

— INFO ;

— WARNING ;

— CRITICAL.

Les statuts possibles peuvent être :

— ACTIVE ;

— READ ;

— RESOLVED.

Ces valeurs constituent des propositions de conception à harmoniser avec les énumérations Java du projet.

4.5 Règles métier principales

Le microservice doit respecter les règles métier suivantes :

1. Seul un utilisateur autorisé peut consulter ou modifier les relevés de son foyer.
2. Chaque relevé doit être associé à un foyer existant et actif.
3. Les quantités de production et de consommation doivent être positives ou nulles.
4. Un relevé doit contenir une date valide.
5. Un foyer ne peut pas posséder deux relevés pour la même date.
6. La date de début d'un bilan ne doit pas être postérieure à sa date de fin.
7. Les bilans doivent être calculés à partir des relevés associés au foyer concerné.
8. Le surplus ne doit jamais être négatif.
9. Toute modification ou suppression d'un relevé doit être prise en compte dans les bilans concernés.
10. Les alertes doivent être générées selon des conditions métier clairement définies.
11. Les données énergétiques ne doivent pas être accessibles aux utilisateurs non autorisés.

Précision :
Le calcul du surplus reste une estimation théorique, fondée sur les données saisies manuellement par les utilisateurs.
Ce calcul ne représente pas une mesure certifiée du réseau électrique.

4.6 Architecture interne Spring Boot

Le **Energy Monitoring Service** repose sur une architecture logicielle en couches développée avec **Spring Boot**.

Cette organisation permet de séparer les responsabilités techniques de la logique métier.

Les principaux composants sont :

— **REST Controllers** : réception et traitement des requêtes HTTP.

— **Service Layer** : gestion des relevés, calcul des bilans et génération des alertes.

— **Spring Data JPA Repositories** : accès aux données relationnelles.

— **PostgreSQL** : stockage des relevés, des bilans et des alertes.

— **OpenFeign Clients** : vérification des foyers auprès du Household Service.

Le frontend React communique avec le microservice à travers l'API Gateway.

La couche Service effectue les traitements métier avant d'accéder aux repositories.

Le microservice est également consulté par le **Energy Marketplace Service**, notamment lors de la vérification du surplus énergétique déclaré.

4.6.1 Diagramme d'architecture technique

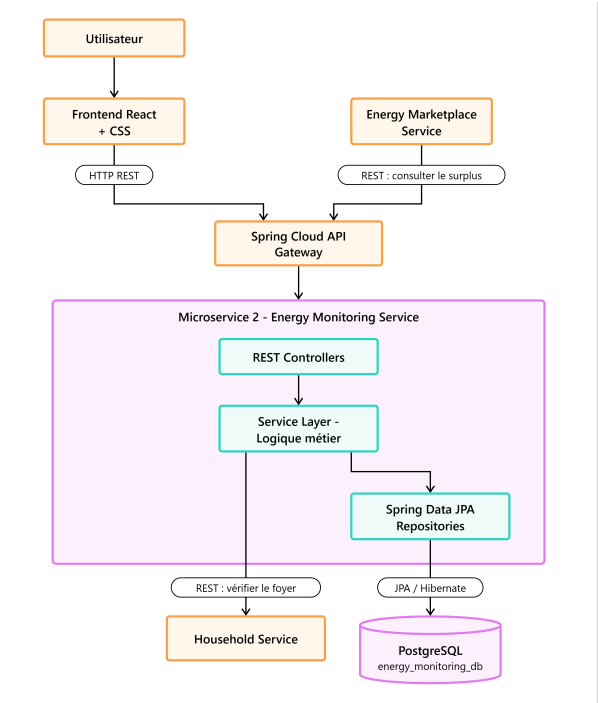


FIGURE 4.2 – Architecture technique de monitoring service

Architecture technique à insérer :
Ajouter le fichier `images/microservice2_architecture.png` dans le projet Overleaf.

4.7 Contrats API REST prévus

Le microservice expose des endpoints REST permettant de gérer les relevés, les bilans énergétiques, les statistiques et les alertes.

Ces API sont accessibles depuis l'interface React via l'API Gateway.

Méthode	Endpoint	Fonction	Statut
POST	`/api/readings`	Enregistrer un relevé énergétique.	201
GET	`/api/readings`	Consulter les relevés d'un foyer.	200
GET	`/api/readings/{id}`	Consulter un relevé spécifique.	200
PUT	`/api/readings/{id}`	Modifier un relevé existant.	200
DELETE	`/api/readings/{id}`	Supprimer un relevé énergétique.	204
GET	`/api/balances/{householdId}`	Consulter le bilan et le surplus d'un foyer.	200
GET	`/api/statistics/{householdId}`	Consulter les statistiques énergétiques.	200
GET	`/api/alerts`	Consulter les alertes énergétiques.	200
PATCH	`/api/alerts/{id}/status`	Mettre à jour le statut d'une alerte.	200

4.7.1 Codes HTTP et gestion des erreurs

Les principaux codes HTTP prévus pour le microservice sont :

— `200 OK` : consultation ou modification réussie.

— `201 Created` : création d'un relevé réussie.

— `204 No Content` : suppression réussie.

— `400 Bad Request` : données invalides ou période incohérente.

— `401 Unauthorized` : utilisateur non authentifié.

— `403 Forbidden` : accès non autorisé au foyer.

— `404 Not Found` : ressource demandée introuvable.

— `409 Conflict` : relevé déjà existant pour la même date.

— `503 Service Unavailable` : dépendance externe temporairement indisponible.

Les endpoints présentés constituent des **contrats de conception** à implémenter et tester dans Spring Boot.

4.8 Communication inter-microservices

Le **Energy Monitoring Service** communique avec les autres microservices selon leurs responsabilités métier.

La communication principale repose sur des appels REST synchrones.

4.8.1 Communication avec Household Service

Le Monitoring peut utiliser **OpenFeign** pour vérifier l'existence et l'état d'un foyer.
Cette vérification est notamment utile avant l'enregistrement d'un relevé énergétique.
Le Household Service reste le propriétaire des informations relatives aux foyers.
Le Monitoring conserve seulement l'identifiant du foyer nécessaire à ses traitements.

4.8.2 Communication avec Marketplace Service

Le **Energy Marketplace Service** doit pouvoir consulter les informations de surplus énergétique avant de permettre la publication d'une offre.

La communication est synchrone, via une API REST exposée par le Monitoring.
Le Marketplace utilise un **OpenFeign Client** pour interroger cette API.
Cette communication permet notamment de :

— récupérer le bilan énergétique d'un foyer ;

— consulter le surplus théorique calculé pour une période ;

— vérifier la cohérence de la quantité proposée dans une offre.

Le Monitoring ne crée pas l'offre énergétique.
Il fournit uniquement les données utilisées par le Marketplace dans sa logique de validation.

4.8.3 Synthèse des interactions

Service	Mode	Rôle de la communication
Household Service	OpenFeign / REST	Vérification de l'existence et de l'état d'un foyer avant certaines opérations.
Marketplace Service	REST synchrone	Consultation des bilans et du surplus énergétique déclaré pour la validation des offres.
Transaction Service	Aucun appel direct requis	La gestion des transactions reste indépendante du suivi énergétique.
RAG Assistant	Aucun appel direct requis	Le RAG utilise une base documentaire et ne calcule pas les bilans du Monitoring.

4.9 Événements RabbitMQ

L'architecture globale EnergyShare utilise **RabbitMQ** pour permettre des communications asynchrones entre certains microservices.

Cependant, dans la conception actuelle, le Monitoring ne dépend pas obligatoirement de RabbitMQ pour remplir ses fonctionnalités principales.

La consultation des surplus par le Marketplace repose sur une communication **REST synchrone via OpenFeign**.

Une évolution possible consiste à publier des événements lorsqu'un relevé ou un bilan énergétique est mis à jour.

4.9.1 Événements envisageables

— `EnergyReadingCreated` : notification après l'enregistrement d'un relevé.

— `EnergyReadingUpdated` : notification après la modification d'un relevé.

— `EnergyBalanceUpdated` : notification lorsqu'un bilan énergétique est recalculé.

— `EnergyAlertCreated` : notification après la création d'une alerte.

Ces événements sont proposés comme possibilités d'évolution et ne constituent pas des dépendances obligatoires du microservice Monitoring.

Leurs contrats devront être définis avec les éventuels services consommateurs avant toute implémentation.

4.10 Résilience et cohérence des données

Le microservice doit garantir la fiabilité de ses traitements et la cohérence des données énergétiques enregistrées.

4.10.1 Validation des données

Les valeurs de production et de consommation doivent être vérifiées avant leur stockage.

Une quantité négative ou une date invalide entraîne le rejet de la requête.

L'unicité des relevés quotidiens doit également être assurée par une contrainte PostgreSQL.

4.10.2 Gestion de l'indisponibilité des services

Si le Household Service est temporairement indisponible, une opération nécessitant la validation d'un foyer ne doit pas être acceptée sans vérification.

Le système retourne alors une réponse d'erreur adaptée.

Des mécanismes de timeout et de gestion des exceptions peuvent être configurés dans les clients OpenFeign.

4.10.3 Cohérence des bilans

Lorsqu'un relevé énergétique est modifié ou supprimé, les bilans concernés doivent refléter les nouvelles données.

Deux stratégies sont possibles :

— recalculer le bilan à chaque consultation ;

— conserver les bilans et les actualiser lorsque les relevés changent.

Dans le deuxième cas, la mise à jour doit être réalisée de manière cohérente, par exemple dans une transaction applicative.

4.10.4 Sécurité des données

L'accès aux données énergétiques doit être protégé par l'authentification **Keycloak**.

Le microservice doit vérifier l'identité de l'utilisateur et son droit d'accès au foyer demandé.

La présence d'un identifiant `householdId` dans une requête ne suffit pas à autoriser l'accès aux informations correspondantes.

4.11 Conclusion

Le microservice **Energy Monitoring Service** occupe une place essentielle dans l'architecture EnergyShare.

Il centralise les informations relatives à la production et à la consommation électrique des foyers, calcule leurs bilans énergétiques et estime les surplus théoriques.

Grâce à **Spring Boot**, **Spring Data JPA** et **PostgreSQL**, il repose sur une architecture modulaire et indépendante.

Son interaction avec le **User & Household Service** permet de vérifier les foyers, tandis que ses API REST fournissent au **Energy Marketplace Service** les informations nécessaires à la validation des offres.

Il constitue ainsi la base fonctionnelle du suivi énergétique de la plateforme et contribue à une meilleure visibilité des ressources solaires déclarées par les foyers tunisiens.

Chapitre 5

Microservice 3 – Energy Marketplace Service

5.1 Positionnement et périmètre

Energy Marketplace Service constitue le contexte métier dédié aux offres de surplus solaire, demandes d'énergie, recherches et réservations. Il met en relation producteurs et consommateurs sans effectuer de mesures électriques ni gérer les paiements. Son périmètre reste distinct de Household (foyers), Monitoring (relevés) et Transaction (exécutions et paiements simulés). Le RAG est un assistant documentaire séparé et n'intervient pas dans le matching de base.

5.2 Fonctionnalités détaillées

5.2.1 Gestion des offres

Le producteur peut créer une offre mentionnant le foyer vendeur, la quantité proposée, le prix par kWh, la zone et la période de disponibilité. Il peut consulter, filtrer et modifier ses offres selon les conditions prévues, ainsi qu'annuler celles qui ne sont pas engagées dans une transaction incompatible.

5.2.2 Gestion des demandes

Le consommateur exprime un besoin : quantité souhaitée, prix maximal, période et foyer acheteur. La demande peut être ouverte, partiellement satisfaite, satisfaite ou annulée.

5.2.3 Recherche et matching

Le service filtre les offres selon le statut, la période, la quantité réservable, la localisation et le prix. Le matching reste **déterministe** : ces règles métier ne sont pas présentées comme un algorithme d'apprentissage automatique.

5.2.4 Gestion des réservations

Une réservation bloque temporairement une quantité sur une offre. Elle est confirmée ou annulée selon le résultat transmis par le service Transaction, ou libérée à expiration. Le système évite les doubles réservations sur la même quantité.

5.3 Modèle de données MongoDB

Collection	Champs importants	Rôle
`energy_offers`	`id`, `sellerHouseholdId`, `quantityKwh`, `remainingKwh`, `pricePerKwh`, `zone`, `startTime`, `endTime`, `status`	Stockage des offres et de leur disponibilité.
`energy_requests`	`id`, `buyerHouseholdId`, `requestedKwh`, `maxPrice`, `startTime`, `endTime`, `status`	Besoins des consommateurs.
`energy_reservations`	`id`, `offerId`, `requestId`, `buyerHouseholdId`, `reservedKwh`, `expiresAt`, `status`	Réservations rattachées à une offre et éventuellement à une demande.

5.3.1 Relations et intégrité

Une offre peut être concernée par plusieurs réservations ; chaque réservation concerne exactement une offre. Une demande peut donner lieu à plusieurs réservations ; une réservation peut également être créée sans demande préalable. Les associations sont réalisées avec les identifiants MongoDB et des contrôles applicatifs. Le champ `sellerHouseholdId` désigne le foyer externe sans relation de base de données interservices.

5.4 Diagramme de classes du microservice 3

5.4.1 Présentation du modèle de données

Le microservice **Energy Marketplace Service** repose sur un modèle de données organisé autour de trois entités métier principales :

— **EnergyOffer** : gestion des offres de surplus d'énergie solaire.

— **EnergyRequest** : gestion des demandes énergétiques des consommateurs.

— **EnergyReservation** : gestion des réservations effectuées sur les offres.

Ces entités sont stockées dans une base de données **MongoDB**, propre au microservice Marketplace.

Le modèle respecte le principe *Database per Service*, selon lequel chaque microservice possède et gère indépendamment ses propres données.

5.4.2 Représentation UML

La figure 5.1 présente le diagramme de classes du **Energy Marketplace Service**.

Elle illustre les attributs principaux des entités et les relations entre les offres, les demandes et les réservations énergétiques.

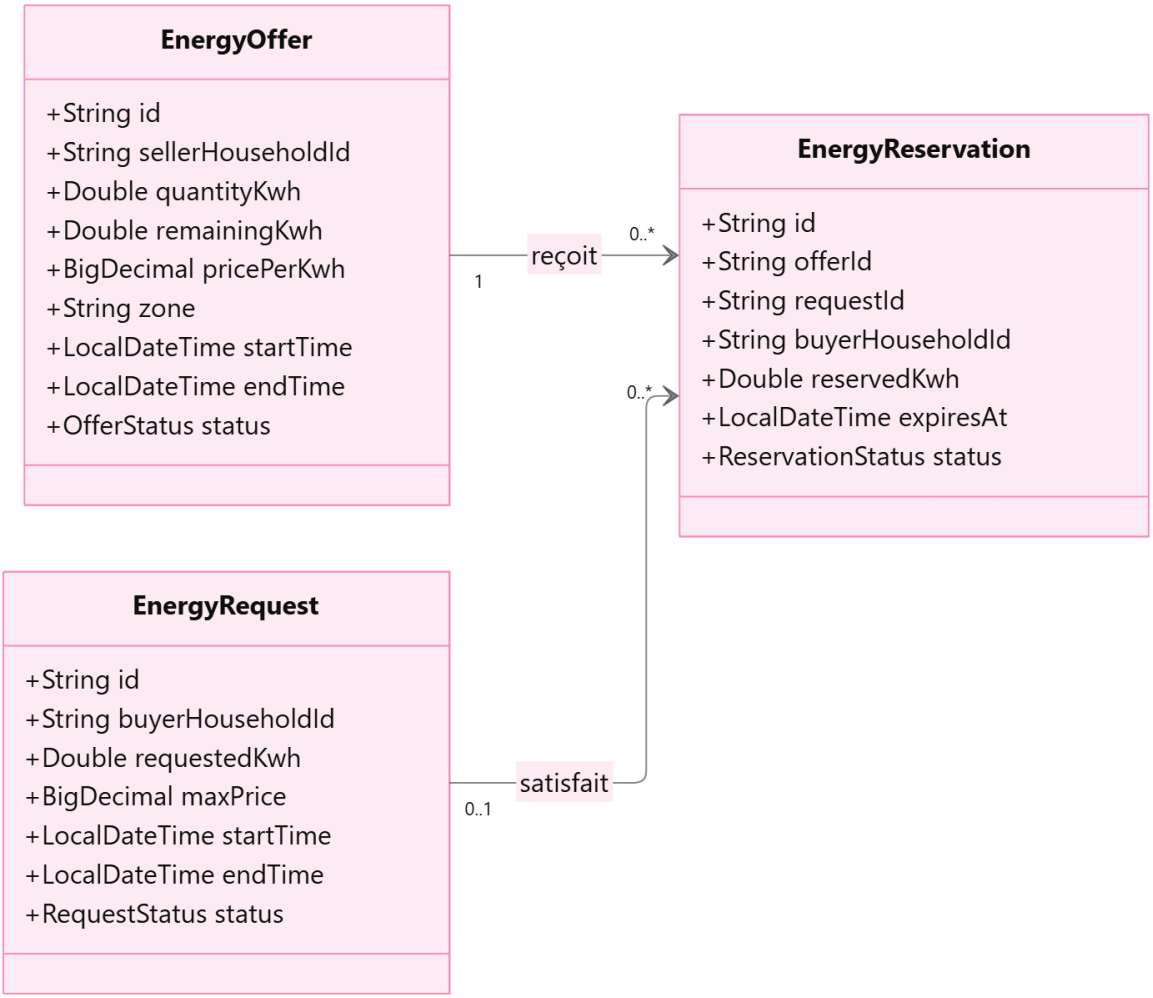


FIGURE 5.1 – Diagramme de classes du Energy Marketplace Service.

5.4.3 Description des entités

Entité EnergyOffer

L'entité **EnergyOffer** représente une offre d'énergie solaire publiée par un foyer producteur sur la plateforme EnergyShare.

Elle contient les informations nécessaires à la publication et à la gestion d'une offre.

— **id** : identifiant unique de l'offre.

— **sellerHouseholdId** : identifiant du foyer producteur.

— **quantityKwh** : quantité totale d'énergie proposée, exprimée en kilowattheures (kWh).

— **remainingKwh** : quantité d'énergie encore disponible pour les réservations.

— **pricePerKwh** : prix unitaire simulé de l'énergie proposée.

— **status** : état de l'offre, par exemple ACTIVE, EXPIRED ou CANCELLED.

Exemple d'offre énergétique

Un foyer producteur publie une offre de **8 kWh** au prix simulé de **0,25 DT/kWh**. Après une réservation de 5 kWh, la quantité encore disponible est de **3 kWh**.

Entité EnergyRequest

L'entité **EnergyRequest** représente une demande d'énergie créée par un foyer consommateur.

Elle permet aux utilisateurs d'exprimer leurs besoins énergétiques afin de rechercher des offres compatibles.

— **id** : identifiant unique de la demande.

— **buyerHouseholdId** : identifiant du foyer consommateur.

— **requestedKwh** : quantité d'énergie souhaitée.

— **maxPrice** : prix maximal accepté par le consommateur.

— **status** : état de la demande, par exemple OPEN, PARTIALLY_FULFILLED, FULFILLED ou CANCELLED.

Le Marketplace utilise ces informations pour rechercher les offres correspondant aux critères du consommateur, notamment la quantité, le prix et la disponibilité.

Entité EnergyReservation

L'entité **EnergyReservation** représente une réservation effectuée par un consommateur sur une offre énergétique disponible.

Elle permet de bloquer temporairement une quantité d'énergie et de suivre l'évolution de la réservation.

— **id** : identifiant unique de la réservation.

— **offerId** : identifiant de l'offre concernée.

— **requestId** : identifiant de la demande associée, lorsqu'elle existe.

— **reservedKwh** : quantité d'énergie réservée.

— **status** : état de la réservation, par exemple PENDING, CONFIRMED, CANCELLED ou EXPIRED.

Une réservation peut être confirmée après validation de la transaction simulée, annulée par l'utilisateur ou expirée lorsque son délai de validité est dépassé.

5.4.4 Relations entre les entités

Relation EnergyOffer – EnergyReservation

Une offre énergétique peut être associée à plusieurs réservations.

Chaque réservation concerne cependant une seule offre.

La cardinalité est donc :

$$\textbf{EnergyOffer (1)} \longrightarrow \textbf{(0..*) EnergyReservation}$$

Cette relation permet de gérer les réservations partielles d'une même offre et de suivre la quantité d'énergie restante.

Relation EnergyRequest – EnergyReservation

Une demande énergétique peut être satisfaite par plusieurs réservations.
Une réservation peut également être créée directement depuis une offre, sans demande préalable.
La cardinalité est donc :

EnergyRequest (0..1) ⟷ (0..*) EnergyReservation

Chaque réservation est ainsi associée à zéro ou une demande, tandis qu'une demande peut donner lieu à plusieurs réservations.

5.4.5 Cohérence et intégrité des données

Le modèle de données doit respecter plusieurs contraintes métier :

— Une offre ne peut pas proposer une quantité d'énergie négative ou nulle.

— Une réservation ne peut pas dépasser la quantité disponible dans l'offre concernée.

— Une offre expirée ou annulée ne peut plus recevoir de réservations.

— Lorsqu'une réservation est annulée ou expire, la quantité bloquée doit être libérée une seule fois.

— Plusieurs réservations simultanées ne doivent pas entraîner de surréservation énergétique.

— Les identifiants des foyers sont obtenus auprès du User & Household Service.

La cohérence des réservations sera notamment assurée par des opérations atomiques conditionnelles sur MongoDB, afin d'éviter que plusieurs utilisateurs réservent simultanément la même quantité.

Remarque : Les relations entre les collections du Marketplace sont représentées à l'aide d'identifiants et contrôlées par la logique métier.
Le microservice ne possède aucune relation directe avec les bases de données des autres microservices.

5.5 Règles métier principales

1. Seul un foyer actif et autorisé peut proposer une offre.
2. La quantité proposée et le prix doivent être strictement positifs.
3. La période de disponibilité doit être cohérente (début antérieur à fin).
4. La publication doit reposer sur une vérification du surplus déclaré pour la période considérée.
5. La quantité cumulée des réservations actives ne doit jamais dépasser la quantité de l'offre.
6. Une offre expirée ou annulée n'est pas réservable.
7. Une réservation expirée ou annulée doit libérer la quantité bloquée, sans double libération.
8. Les opérations sensibles doivent être idempotentes ou protégées contre les doubles soumissions.

5.6 Architecture interne Spring Boot

Le **Energy Marketplace Service** repose sur une architecture en couches. Les **Controllers REST** reçoivent les requêtes HTTP, les **Services** appliquent les règles métier et les **Repositories Spring Data MongoDB** assurent l'accès aux données.

Le microservice utilise également des **Feign Clients** pour les validations auprès des services Household et Monitoring, ainsi que des composants **RabbitMQ** pour les communications asynchrones avec le Transaction Service.

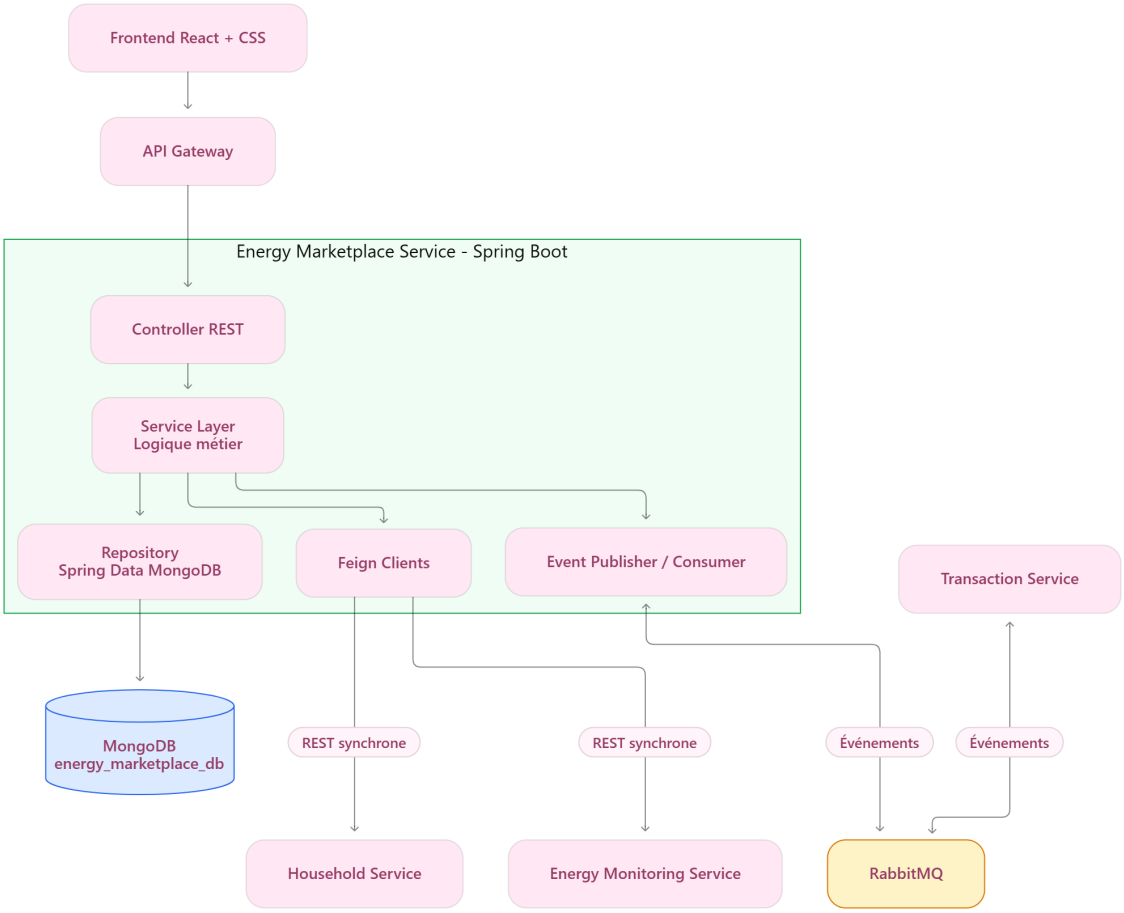


FIGURE 5.2 – Architecture interne du microservice 3 — Energy Marketplace Service.

5.7 Contrats API REST prévus

Méthode	URI	Fonction	Succès
POST	`/api/offers`	Publier une offre	201
GET	`/api/offers`	Rechercher des offres	200
GET	`/api/offers/{id}`	Consulter une offre	200
PUT	`/api/offers/{id}`	Modifier une offre autorisée	200
POST	`/api/requests`	Créer une demande	201
GET	`/api/requests`	Consulter les demandes	200
POST	`/api/reservations`	Réserver une quantité	201
GET	`/api/reservations/{id}`	Consulter une réservation	200

Méthode	URI	Fonction	Succès
DELETE	`/api/reservations/{id}`	Annuler une réservation	204

Les réponses d'erreur principales sont : `400` (données invalides), `401` (non authentifié), `403` (droits insuffisants), `404` (ressource introuvable), `409` (conflit de réservation) et `503` (dépendance indisponible). Les ressources et URI ci-dessus sont des **contrats de conception** et non la preuve d'une implémentation terminée.

5.8 Communication inter-microservices

Interlocuteur	Mode	Données et justification
Household	OpenFeign / REST	Vérification de l'existence, du rôle et de l'état du foyer vendeur. Réponse immédiate nécessaire.
Monitoring	OpenFeign / REST	Consultation du surplus déclaré et de sa période pour valider une publication.
Transaction	RabbitMQ	Réservation créée, transaction confirmée ou annulée. Réconciliation asynchrone des états.

5.8.1 Événements prévus

— `ReservationCreated` : Marketplace informe Transaction de la création d'une réservation.

— `TransactionConfirmed` : Transaction confirme un échange ; Marketplace actualise le statut réservé.

— `TransactionCancelled` : Transaction signale une annulation ; Marketplace libère la quantité concernée.

Ces noms et contrats d'événements devront être harmonisés avec le microservice Transaction lors de l'implémentation.

5.9 Résilience et cohérence

En cas d'indisponibilité de Household ou Monitoring, toute opération exigeant une validation externe est refusée temporairement, plutôt que d'utiliser une information incertaine. La réservation concurrente doit s'appuyer sur des **mises à jour atomiques conditionnelles MongoDB** (vérification de quantité restante). Pour RabbitMQ, les messages sont traités de manière **idempotente** ; des reprises et une stratégie de publication fiable (par exemple *outbox*) sont envisagées. L'expiration des réservations doit libérer une quantité une seule fois.

Chapitre 6

Microservice 4 – Energy Transaction Service

6.1 Positionnement et périmètre

Le **Energy Transaction Service** est le microservice chargé du cycle de vie des échanges d'énergie entre un foyer **vendeur** (producteur) et un foyer **acheteur** (consommateur) sur la plateforme **EnergyShare**.

Il intervient **après** la réservation d'une offre dans le Marketplace : il vérifie cette réservation, calcule le montant de l'échange, simule le paiement, suit l'état de la transaction, conserve son historique et délivre un justificatif d'échange.

Il est le **propriétaire** des transactions, des paiements simulés et de l'historique des statuts. Les autres microservices n'en conservent que l'identifiant ou en reçoivent le résultat par événement.

Le périmètre de ce microservice est distinct de celui des autres services :

— **User & Household Service** : gestion des utilisateurs, des foyers et des installations.

— **Energy Monitoring Service** : relevés, bilans et surplus déclarés.

— **Energy Marketplace Service** : offres, demandes et réservations. C'est lui qui détient la réservation à l'origine de chaque transaction.

— **EnergyShare RAG Assistant** : assistance documentaire, sans lien avec les transactions.

Le Transaction Service ne crée ni offre, ni demande, ni réservation, et ne calcule aucun surplus.

Remarque :
Dans ce prototype académique, le paiement est une **simulation logicielle** (méthode `SIMULATED`). Aucune somme réelle n'est débitée, aucune banque ni passerelle de paiement n'est contactée, et aucune électricité n'est physiquement livrée : une transaction représente un échange simulé.

6.2 Fonctionnalités détaillées

6.2.1 Création et validation d'une transaction

L'acheteur crée une transaction en indiquant uniquement l'identifiant de sa réservation. Le service interroge alors le Marketplace et vérifie que la réservation :

— existe et est encore active (statut `PENDING`) ;

— appartient bien à l'acheteur authentifié ;

— n'a pas déjà donné lieu à une transaction.

Le foyer vendeur, la quantité et le prix par kWh sont **lus auprès du Marketplace** et jamais fournis par le client : cela empêche un utilisateur de modifier le montant d'un échange ou d'usurper un foyer.

Le montant total est ensuite calculé automatiquement :

$$M = Q \times P$$

Avec :

— M : montant total de la transaction (DT), arrondi à deux décimales ;

— Q : quantité réservée (kWh) ;

— P : prix unitaire simulé (DT/kWh).

Exemple de transaction

Une réservation de **5 kWh** sur une offre à **0,25 DT/kWh** donne une transaction de montant **1,25 DT**. L'offre de 8 kWh du chapitre précédent conserve alors 3 kWh disponibles.

6.2.2 Paiement simulé et confirmation

L'acheteur déclenche le paiement d'une transaction en attente. Le simulateur de paiement répond de deux façons possibles :

— **Paiement accepté** : le paiement est enregistré avec le statut `SUCCESS`, la transaction passe automatiquement à `CONFIRMED` et le Marketplace est informé pour confirmer la réservation.

— **Paiement refusé** : le paiement est enregistré avec le statut `FAILED`, la transaction passe à `FAILED` et le Marketplace est informé pour **libérer la réservation** et restituer la quantité à l'offre.

Une panne **technique** du simulateur n'est pas un refus : la transaction reste alors en attente et le paiement peut être retenté.

6.2.3 Clôture et annulation

Une fois l'énergie livrée, le foyer **vendeur** clôture l'échange : la transaction passe de `CONFIRMED` à `COMPLETED` et la date de clôture est enregistrée. Dans cette version, la clôture est déclenchée par un appel API ; elle pourrait à terme s'appuyer sur les relevés du Monitoring.

L'acheteur ou le vendeur peut annuler une transaction tant qu'elle est `PENDING` ou `CONFIRMED`. Si elle était déjà payée, le paiement est marqué `REFUNDED` (remboursement simulé). Une transaction qui n'est pas payée dans un délai de 15 minutes (paramétrable) est annulée automatiquement.

6.2.4 Suivi, historique et justificatif

Les utilisateurs concernés peuvent :

— consulter une transaction et son statut courant ;

— lister leurs transactions, avec filtres et pagination ;

— consulter l'historique complet des changements de statut ;

— obtenir un justificatif d'échange pour une transaction terminée.

6.3 Modèle de données PostgreSQL

Le microservice utilise **PostgreSQL** (base `transaction_db`) et **Spring Data JPA**. Une transaction gère de l'argent, même simulé, et plusieurs données doivent évoluer ensemble : la transaction, son paiement et son historique. Les propriétés **ACID** de PostgreSQL et ses contraintes d'intégrité (unicité, clés étrangères, contrôles) protègent ces règles même en cas de requêtes concurrentes.

Le modèle comprend trois entités métier :

— **EnergyTransaction**

— **Payment**

— **TransactionHistory**

6.3.1 Structure des tables

Table	Champs principaux	Rôle
`energy_transactions`	`id`, `reservationId`, `sellerHouseholdId`, `buyerHouseholdId`, `quantityKwh`, `unitPrice`, `totalAmount`, `status`, `createdAt`, `completedAt`, `cancellationReason`, `version`	Échange d'énergie issu d'une réservation.
`payments`	`id`, `transactionId`, `amount`, `paymentMethod`, `paymentStatus`, `paymentReference`, `paymentDate`	Paiements simulés associés aux transactions.
`transaction_history`	`id`, `transactionId`, `previousStatus`, `newStatus`, `description`, `changedAt`	Historique des changements de statut.
`outbox_events`	`id`, `aggregateId`, `eventType`, `routingKey`, `payload`, `createdAt`, `publishedAt`	Table technique : événements RabbitMQ à publier.
`processed_events`	`eventId`, `processedAt`	Table technique : événements reçus déjà traités.

Les deux dernières tables ne sont pas des entités métier : elles servent uniquement à fiabiliser la communication asynchrone.

6.3.2 Relations et intégrité des données

— une transaction peut avoir plusieurs paiements dans le modèle, mais une seule tentative est autorisée dans cette version ;

— une transaction possède au moins une ligne d'historique (celle de sa création) ;

— le champ `reservationId` porte une contrainte d'**unicité** : une réservation ne peut donner lieu qu'à une seule transaction, même si deux requêtes arrivent en même temps ;

— un **index unique partiel** interdit deux paiements réussis pour la même transaction ;

— les montants et quantités sont manipulés en `BigDecimal` et stockés en `NUMERIC`, jamais en nombre flottant, pour éviter les erreurs d'arrondi ;

— la colonne `version` assure un **verrou optimiste** : deux modifications simultanées d'une même transaction sont détectées ;

— l'historique et les paiements ne sont jamais supprimés.

Les identifiants `reservationId`, `sellerHouseholdId` et `buyerHouseholdId` sont des **références opaques** vers le Marketplace et le Household Service. Ils ne constituent pas des clés étrangères interservices, conformément au principe *Database per Service*.

6.4 Diagramme de classes du microservice 4

6.4.1 Représentation UML

Le diagramme de classes de l'**Energy Transaction Service** représente les trois entités métier, leurs énumérations et leurs relations.

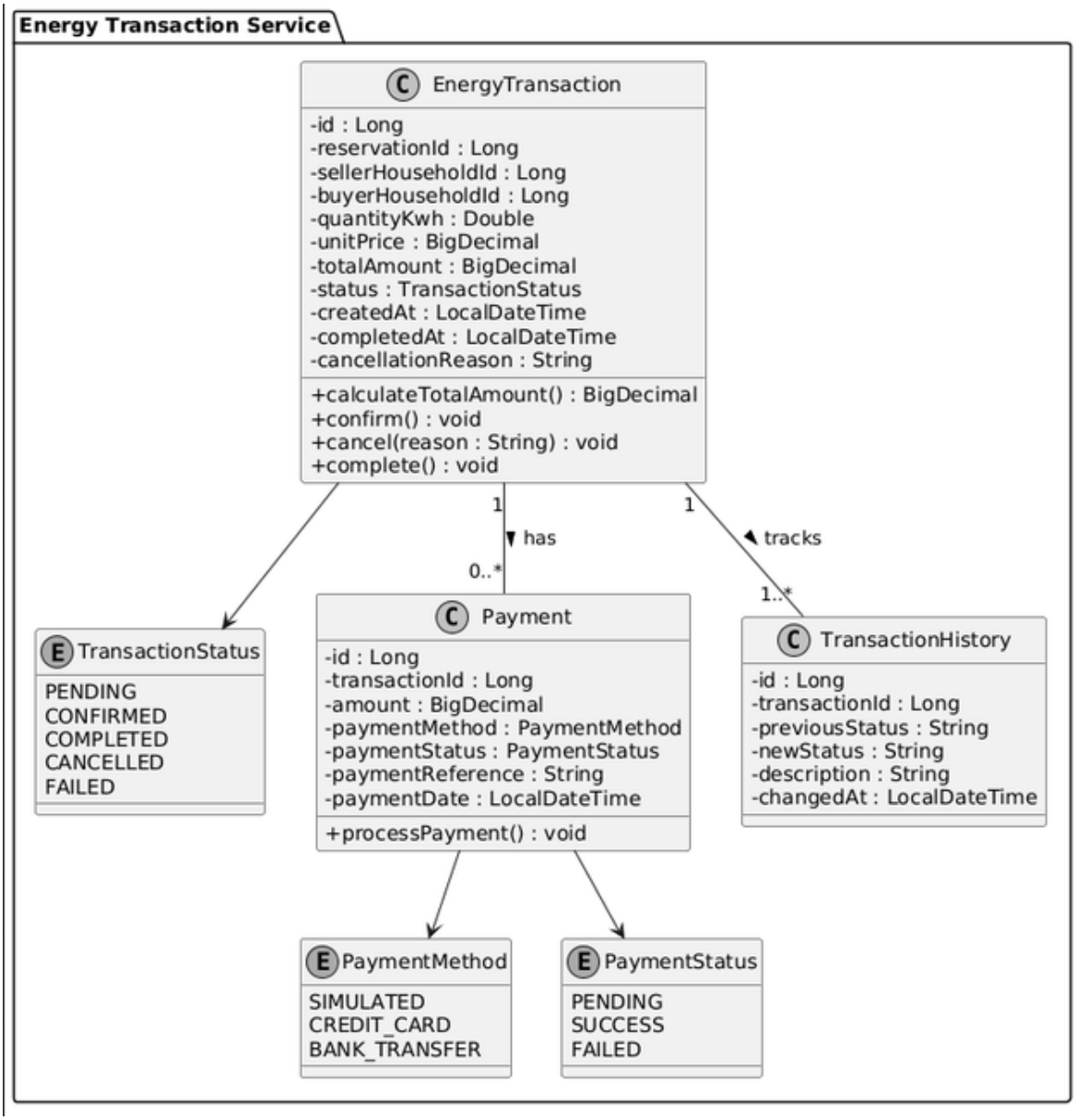


FIGURE 6.1 – Diagramme de classes du Energy Transaction Service.

6.4.2 Description des entités

Entité EnergyTransaction

L'entité **EnergyTransaction** représente un échange d'énergie entre un vendeur et un acheteur.

— **id** : identifiant unique, de type Long.

— **reservationId** : réservation du Marketplace à l'origine de la transaction (unique), de type String.

— **sellerHouseholdId** et **buyerHouseholdId** : foyers vendeur et acheteur, de type String.

— **quantityKwh** : quantité échangée, de type BigDecimal, copiée depuis la réservation.

— **unitPrice** : prix par kWh, de type BigDecimal, copié depuis la réservation.

— **totalAmount** : montant total calculé, de type BigDecimal.

— **status** : statut courant (`TransactionStatus`).

— **createdAt** et **completedAt** : dates de création et de clôture (UTC), de type Instant.

— **cancellationReason** : motif d'annulation ou d'échec.

— **version** : compteur du verrou optimiste.

L'entité porte les opérations métier `calculateTotalAmount()`, `confirm()`, `complete()`, `cancel(reason)` et `fail(reason)`. Chacune refuse les transitions interdites par le cycle de vie.

Les données du vendeur, de la quantité et du prix sont des **copies figées** prises à la création : une modification ultérieure côté Marketplace n'affecte pas la transaction.

Entité Payment

L'entité **Payment** représente une tentative de paiement simulé.

— **id** et **transactionId** : identifiant du paiement et transaction concernée.

— **amount** : montant payé, égal au montant total de la transaction.

— **paymentMethod** : `SIMULATED` (seule valeur utilisée), `CREDIT_CARD` et `BANK_TRANSFER` étant réservées à une évolution.

— **paymentStatus** : `PENDING`, `SUCCESS`, `FAILED` ou `REFUNDED`.

— **paymentReference** : référence simulée, par exemple `SIM-20261109-000001`.

— **paymentDate** : date du traitement.

Entité TransactionHistory

L'entité **TransactionHistory** conserve la trace de chaque changement de statut.

— **id** et **transactionId** : identifiant de la ligne et transaction concernée.

— **previousStatus** : ancien statut (vide à la création).

— **newStatus** : nouveau statut.

— **description** : explication du changement (acteur, référence de paiement, commentaire).

— **changedAt** : date du changement.

Cette table fonctionne en **ajout seul** : aucune ligne n'est modifiée ni supprimée.

Relations entre les entités

EnergyTransaction (1) ⟶ (0..*) Payment

EnergyTransaction (1) ⟶ (1..*) TransactionHistory

6.4.3 Cycle de vie d'une transaction

Une transaction suit les statuts `PENDING`, `CONFIRMED`, `COMPLETED`, `CANCELLED` et `FAILED`. Les statuts `COMPLETED`, `CANCELLED` et `FAILED` sont définitifs.

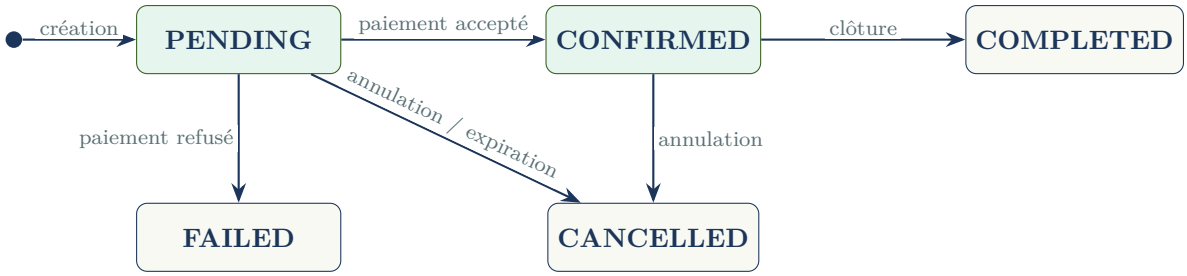


FIGURE 6.2 – Cycle de vie d'une transaction.

De	Vers	Déclencheur	Acteur
(création)	`PENDING`	Création à partir d'une réservation	Acheteur
`PENDING`	`CONFIRMED`	Paiement simulé accepté	Acheteur
`PENDING`	`FAILED`	Paiement simulé refusé	Acheteur
`PENDING`	`CANCELLED`	Annulation, expiration ou réservation annulée	Acheteur, vendeur ou système
`CONFIRMED`	`COMPLETED`	Clôture de l'échange	Vendeur
`CONFIRMED`	`CANCELLED`	Annulation avec remboursement simulé	Acheteur ou vendeur

Toute autre transition est refusée avec une erreur `409 Conflict`.

6.5 Règles métier principales

1. Une réservation ne peut donner lieu qu'à **une seule transaction** (contrainte d'unicité en base).
2. Avant toute création, la réservation est vérifiée auprès du Marketplace : elle doit exister, être au statut `PENDING` et appartenir à l'acheteur authentifié.
3. Le vendeur, la quantité et le prix sont copiés depuis la réservation et ne sont jamais fournis par le client.
4. Le montant total est calculé par $M = Q \times P$, en `BigDecimal`, arrondi à deux décimales. La quantité doit être strictement positive et l'acheteur doit être différent du vendeur.
5. Une transaction suit le cycle `PENDING` → `CONFIRMED` → `COMPLETED`. Les statuts `COMPLETED`, `CANCELLED` et `FAILED` sont définitifs.
6. Le paiement n'est possible que sur une transaction `PENDING`, par l'acheteur, pour le montant exact de la transaction. Un seul paiement réussi est possible par transaction.
7. Un paiement **refusé** met la transaction à `FAILED` et déclenche la libération de la réservation. Une panne technique du simulateur n'est pas un refus : la transaction reste `PENDING`.
8. Un paiement réussi confirme automatiquement la transaction ; la quantité reste réservée.
9. Seul le vendeur peut clôturer une transaction `CONFIRMED`.
10. Une transaction n'est annulable que si elle est `PENDING` ou `CONFIRMED`. Si elle était payée, le paiement passe à `REFUNDED` (remboursement simulé).
11. Une transaction `PENDING` non payée après 15 minutes (délai paramétrable) est annulée automatiquement avec le motif `PAYMENT_TIMEOUT`.
12. Chaque changement de statut, création comprise, ajoute une ligne d'historique dans la même transaction de base de données que le changement lui-même.

13. Un événement déjà traité est ignoré : la réception multiple d'un même événement ne doit jamais créer de doublon ni appliquer deux fois un changement.
14. Un justificatif n'est délivré que pour une transaction `COMPLETED`.
15. Seuls l'acheteur et le vendeur concernés peuvent consulter une transaction, son historique et son justificatif ; l'acheteur seul peut payer, le vendeur seul peut clôturer, et les deux peuvent annuler.

6.6 Architecture interne Spring Boot

Le **Energy Transaction Service** repose sur une architecture en couches développée avec **Spring Boot**.

— **REST Controllers** : réception des requêtes HTTP, validation des données et codes de réponse.

— **Service Layer** : règles métier, cycle de vie des transactions, contrôle des droits d'accès et écriture des événements.

— **Spring Data JPA Repositories** : accès aux données PostgreSQL.

— **PostgreSQL et Flyway** : base de données dédiée, schéma créé par des scripts de migration versionnés.

— **OpenFeign Client** : appel synchrone au Marketplace pour vérifier une réservation.

— **Passerelle de paiement simulée** : composant isolé derrière une interface, remplaçable par un vrai prestataire sans modifier la logique métier.

— **Composants RabbitMQ** : publication des événements (table `outbox_events`) et réception des événements du Marketplace.

— **Tâche planifiée** : expiration des transactions non payées.

— **Spring Security (OAuth2 Resource Server)** : validation des jetons émis par Keycloak.

Le service est enregistré dans **Eureka**, récupère sa configuration auprès du **Config Server** et est exposé au frontend React à travers l'**API Gateway**.

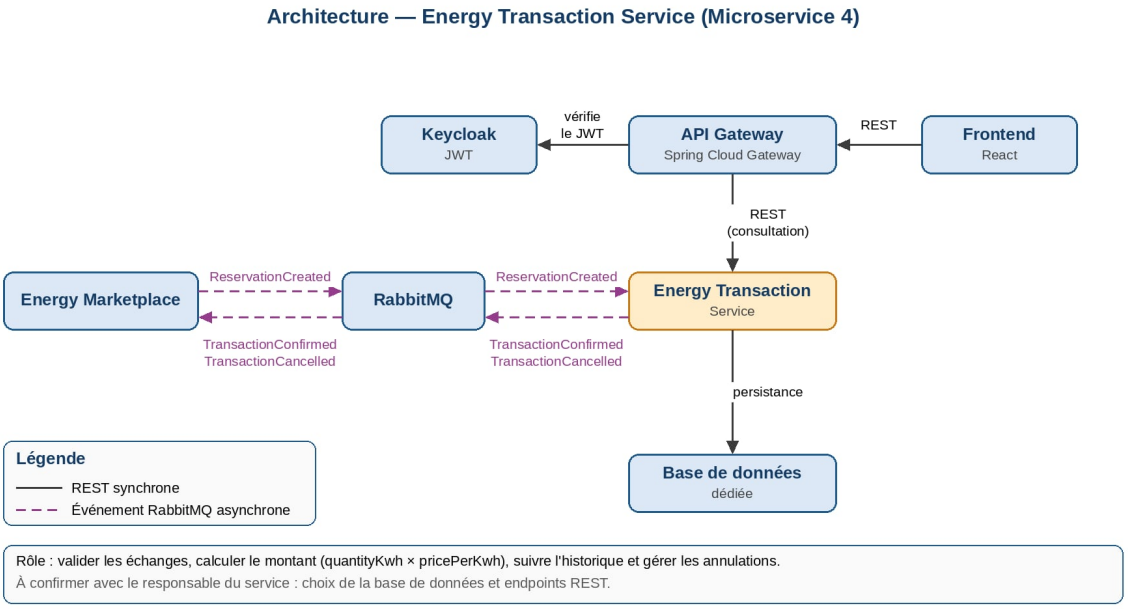


FIGURE 6.3 – Architecture technique du microservice 4 — Energy Transaction Service.

6.7 Contrats API REST prévus

Le microservice expose ses API REST sous le préfixe `/api/transactions`. Elles sont accessibles depuis l'interface React via l'API Gateway.

6.7.1 Correspondance entre fonctionnalités et API

Fonctionnalité	**Réalisation**
F1 – Create Transaction	`POST /api/transactions`
F2 – Validate Transaction	Étape interne de F1 : vérification de la réservation auprès du Marketplace.
F3 – Calculate Amount	Étape interne de F1 : calcul de $M = Q \times P$.
F4 – Process Payment	`POST /api/transactions/{id}/payments`
F5 – Confirm Transaction	Automatique après un paiement réussi ; la clôture se fait avec `PATCH /api/transactions/{id}/complete`.
F6 – Cancel Transaction	`PATCH /api/transactions/{id}/cancel`
F7 – Track Transaction	`GET /api/transactions/{id}`
F8 – Transaction History	`GET /api/transactions` et `GET /api/transactions/{id}/history`
F9 – Generate Receipt	`GET /api/transactions/{id}/receipt`
F10 – Handle Payment Failure	Automatique si le paiement est refusé : transaction `FAILED` et événement `TransactionFailed`.

6.7.2 Endpoints

Acteurs : **A** = acheteur, **V** = vendeur, **Adm** = administrateur (lecture seule).

Méthode	URI	Fonction	Acteurs	Réponses
POST	`/api/transactions`	Créer une transaction à partir d'une réservation	A	201, 400, 401, 403, 404, 409, 503
GET	`/api/transactions/{id}`	Consulter une transaction	A, V, Adm	200, 401, 403, 404
GET	`/api/transactions`	Lister les transactions (filtres, pagination)	A, V, Adm	200, 400, 401, 403
POST	`/api/transactions/{id}/payments`	Payer la transaction (paiement simulé)	A	200, 400, 401, 402, 403, 404, 409, 503
PATCH	`/api/transactions/{id}/complete`	Clôturer l'échange	V	200, 401, 403, 404, 409
PATCH	`/api/transactions/{id}/cancel`	Annuler la transaction	A, V	200, 401, 403, 404, 409
GET	`/api/transactions/{id}/receipt`	Générer le justificatif d'échange	A, V, Adm	200, 401, 403, 404, 409
GET	`/api/transactions/{id}/history`	Consulter l'historique des statuts	A, V, Adm	200, 401, 403, 404

La liste accepte les filtres `status`, `role` (acheteur ou vendeur), `from`, `to`, ainsi que `page`, `size` et `sort`. Un utilisateur ne voit que les transactions de son propre foyer. Il n'existe aucun endpoint de suppression : les transactions, paiements et historiques sont conservés.

6.7.3 Exemples de requêtes et de réponses

Création d'une transaction : l'acheteur n'envoie que l'identifiant de la réservation.

```
POST /api/transactions
{ "reservationId": "res_1001" }

201 Created
{
  "id": 1,
  "reservationId": "res_1001",
  "sellerHouseholdId": "house_101",
  "buyerHouseholdId": "house_202",
  "quantityKwh": 5.0,
  "unitPrice": 0.25,
  "totalAmount": 1.25,
  "status": "PENDING",
  "createdAt": "2026-11-09T12:00:00Z"
}
```

Paiement simulé accepté : la transaction passe à `CONFIRMED`.

```
POST /api/transactions/1/payments

200 OK
{
  "paymentId": 1,
  "transactionId": 1,
  "amount": 1.25,
  "paymentMethod": "SIMULATED",
  "paymentStatus": "SUCCESS",
  "paymentReference": "SIM-20261109-000001",
  "transactionStatus": "CONFIRMED"
}
```

Un paiement refusé renvoie `402 Payment Required` avec le code d'erreur `PAYMENT_REFUSED` ; la transaction est alors `FAILED`.

6.7.4 Codes HTTP et gestion des erreurs

— `200 OK` : consultation ou opération réussie.

— `201 Created` : transaction créée.

— `400 Bad Request` : données invalides.

— `401 Unauthorized` : authentification absente ou invalide.

— `402 Payment Required` : paiement simulé refusé.

— `403 Forbidden` : l'utilisateur n'est ni l'acheteur ni le vendeur concerné, ou n'a pas le rôle requis pour l'action.

— `404 Not Found` : transaction ou réservation introuvable.

— `409 Conflict` : conflit métier (réservation déjà utilisée ou non active, transition de statut interdite, modification concurrente).

— `503 Service Unavailable` : Marketplace ou simulateur de paiement indisponible.

Toutes les erreurs ont le même format JSON : date, code HTTP, code applicatif (par exemple `RESERVATION_ALREADY_USED`), message et chemin de la requête.

6.7.5 Sécurité des accès

Les endpoints exigent un jeton **Keycloak** valide. Le foyer qui agit est déduit du **jeton** (claim `householdId`, hypothèse de conception) et non d'un identifiant fourni dans le corps de la requête : connaître l'identifiant d'un foyer ne suffit pas à agir pour lui. Les montants ne sont jamais fournis par le client.

À harmoniser avec le User & Household Service :
Un utilisateur pouvant appartenir à plusieurs foyers, le lien entre jeton et foyer doit être précisé : claim dédié dans Keycloak, ou vérification d'appartenance auprès du Household Service.

Les endpoints présentés constituent des **contrats de conception** à implémenter et à tester dans Spring Boot.

6.8 Communication inter-microservices

Le Transaction Service dialogue essentiellement avec le **Energy Marketplace Service**, de deux façons complémentaires : un appel REST synchrone pour valider une réservation, et des événements RabbitMQ asynchrones pour propager le résultat de la transaction.

6.8.1 Appel synchrone vers le Marketplace

À la création d'une transaction, le service utilise un **OpenFeign Client** pour interroger le Marketplace sur la réservation concernée. Une réponse immédiate est nécessaire : la transaction ne peut pas être créée sans cette vérification.

Si le Marketplace est indisponible, la création est refusée avec une erreur `503` et **rien n'est enregistré**. Un délai maximal est configuré sur l'appel.

Point à harmoniser avec le Marketplace

Pour créer la transaction, le service a besoin du vendeur, de la quantité réservée et du prix par kWh. La réponse de `GET /api/reservations/{id}` doit donc exposer `buyerHouseholdId`, `sellerHouseholdId`, `reservedKwh`, `pricePerKwh` et `status`. Dans le modèle actuel du Marketplace, le vendeur et le prix figurent sur l'offre : ces champs doivent être recopiés dans la réservation, ou bien le Transaction Service devra interroger aussi `GET /api/offers/{id}`.

6.8.2 Diagrammes de séquence

Le diagramme synchrone décrit la création d'une transaction puis son paiement simulé.

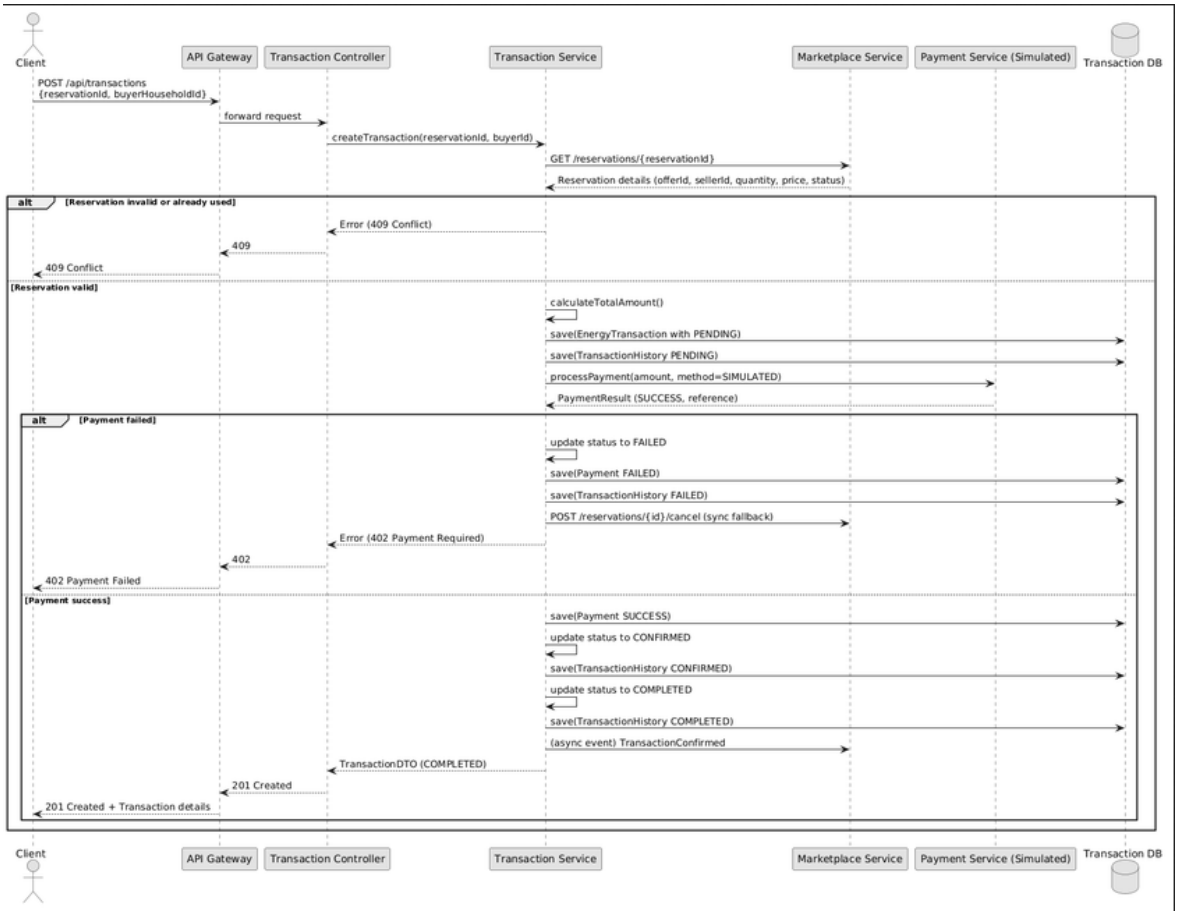


FIGURE 6.4 – Diagramme de séquence synchrone : création et paiement d'une transaction.

Le diagramme asynchrone décrit la propagation des résultats par RabbitMQ et la réaction à une annulation de réservation.

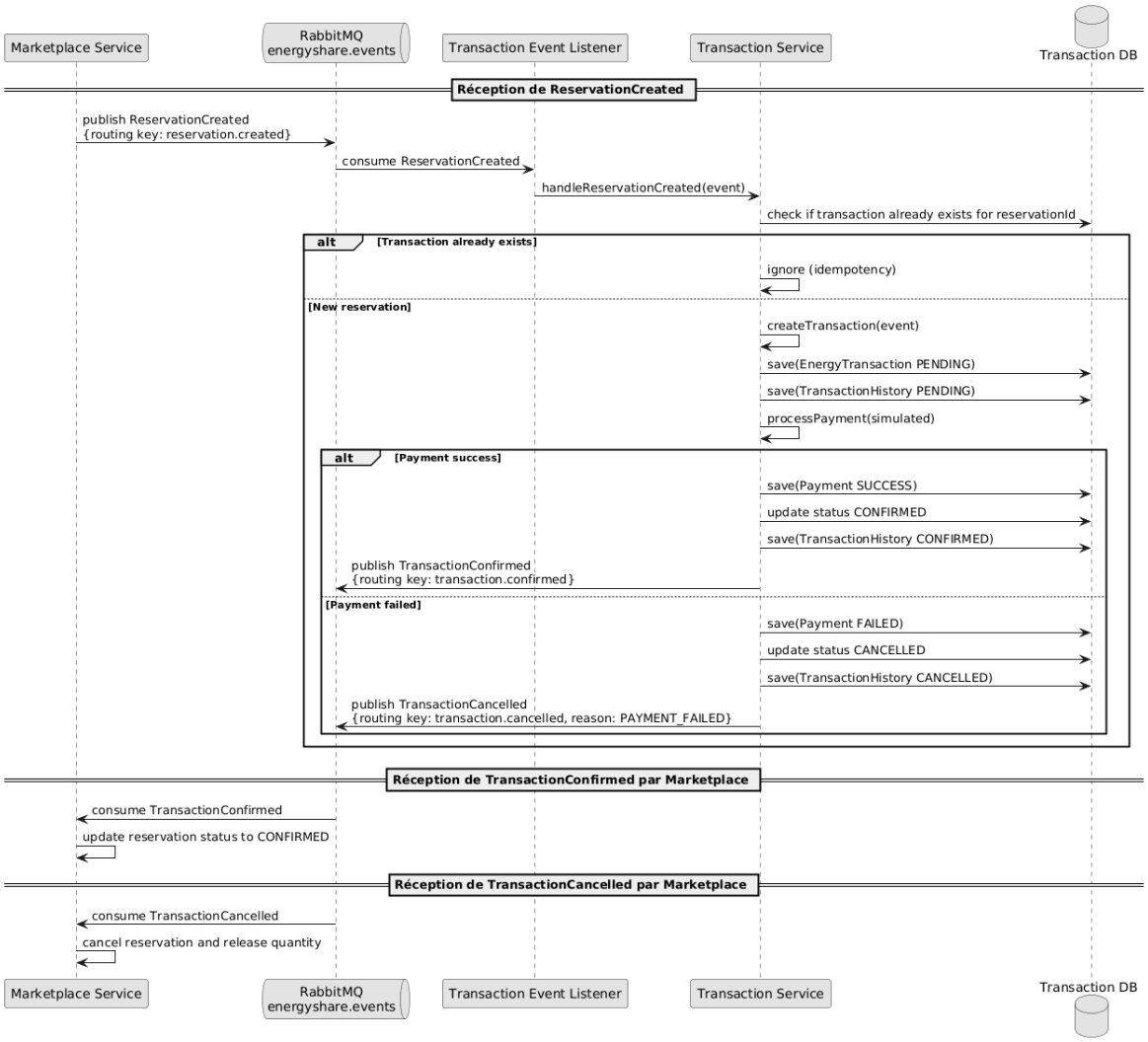


FIGURE 6.5 – Diagramme de séquence asynchrone : événements RabbitMQ entre Transaction et Marketplace.

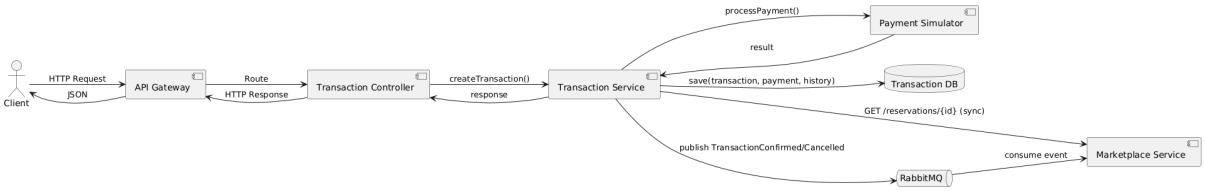


FIGURE 6.6 – Communication du Energy Transaction Service avec les autres microservices.

6.8.3 Synthèse des interactions

Service	Mode	Rôle de la communication
Energy Marketplace Service	OpenFeign / REST et RabbitMQ	Vérification d'une réservation à la création ; événements de confirmation, d'annulation et d'échec ; réception des annulations de réservation.
User & Household Service	Aucun appel direct imposé	Les foyers acheteur et vendeur proviennent de la réservation et du jeton ; une vérification auprès du Household reste envisageable.
Energy Monitoring Service	Aucun appel direct requis	Les transactions sont indépendantes du suivi énergétique.
RAG Assistant	Aucun appel direct requis	L'assistant documentaire n'intervient pas dans les transactions.

6.9 Événements RabbitMQ

La création et le paiement d'une transaction sont synchrones. **RabbitMQ** sert à **propager le résultat** aux autres services sans les bloquer et sans couplage direct. Les messages transitent par l'exchange `energyshare.events`, de type `topic`, avec une `routing key` par type d'événement.

Cette approche correspond à une **saga chorégraphiée** : chaque service réagit aux événements des autres, et la compensation d'un paiement refusé consiste à libérer la réservation.

6.9.1 Événements publiés par le Transaction Service

Événement	Routing key	Effet attendu
`TransactionConfirmed`	`transaction.confirmed`	Le Marketplace passe la réservation de `PENDING` à `CONFIRMED` ; la quantité reste réservée.
`TransactionCancelled`	`transaction.cancelled`	Le Marketplace annule la réservation et restitue la quantité à l'offre.
`TransactionFailed`	`transaction.failed`	Le Marketplace libère la réservation après un paiement refusé.
`TransactionCompleted`	`transaction.completed`	Événement informatif ; aucun consommateur obligatoire dans la conception actuelle.

Exemple de message `TransactionConfirmed` :

```
{
  "eventId": "b7e4c1a2-5f3d-4e8a-9c21-0d6f7a1b2c3d",
  "eventType": "TransactionConfirmed",
  "eventVersion": 1,
  "occurredAt": "2026-11-09T12:01:00Z",
  "transactionId": 1,
  "reservationId": "res_1001",
  "sellerHouseholdId": "house_101",
  "buyerHouseholdId": "house_202",
  "quantityKwh": 5.0,
  "totalAmount": 1.25,
  "status": "CONFIRMED"
}
```

Chaque message porte un identifiant unique `eventId`, un type, une version de schéma et une date.

6.9.2 Événement consommé

Le service consomme l'événement `ReservationCancelled` (`routing key` `reservation.cancelled`), publié par le Marketplace lorsqu'une réservation est annulée ou expire avant confirmation :

— si la transaction est encore `PENDING`, elle passe à `CANCELLED` (motif `RESERVATION_EXPIRED` ou `RESERVATION_CANCELLED`) et l'historique est complété ; aucun événement n'est republié, le Marketplace étant déjà à l'origine de l'annulation ;

— si la transaction est déjà payée ou terminée, l'événement est ignoré et une alerte est journalisée : une réservation payée ne doit pas être annulée par le Marketplace.

À harmoniser avec le chapitre Marketplace :
Le chapitre précédent prévoit l'événement `ReservationCreated`, envoyé par le Marketplace au Transaction Service. Dans la conception retenue ici, la transaction est créée par un **appel REST de l'acheteur** (le paiement exige son action) : `ReservationCreated` n'est donc pas nécessaire. En contrepartie, le Marketplace doit publier `ReservationCancelled` et consommer `TransactionConfirmed`, `TransactionCancelled` et `TransactionFailed` (ou traiter l'échec de paiement comme une annulation avec motif).

6.9.3 Garanties de fiabilité

— **Publication fiable (Transactional Outbox).** Lorsqu'un statut change, l'événement est écrit dans la table `outbox_events` **dans la même transaction de base de données**. Un relais le publie ensuite sur RabbitMQ : si le broker est indisponible, aucun événement n'est perdu.

— **Livraison au moins une fois.** Un message peut être livré plusieurs fois : les consommateurs doivent être idempotents.

— **Idempotence.** Les identifiants des événements reçus sont enregistrés dans `processed_events` ; un doublon est ignoré. Les transitions d'état fournissent une seconde protection.

— **Reprise et messages morts.** Un message en échec est retenté trois fois, puis placé dans une file de messages morts (*Dead Letter Queue*) pour analyse.

— **Ordre.** L'ordre de livraison n'est pas garanti : chaque consommateur décide à partir de l'état courant de l'objet, non de l'ordre d'arrivée.

Ces mécanismes sont prévus dans l'architecture ; les noms et contrats d'événements restent à valider avec le Marketplace avant leur implémentation.

6.10 Résilience et cohérence des données

6.10.1 Validation des données

Les requêtes sont validées avant traitement : identifiant de réservation obligatoire, méthode de paiement autorisée, taille des commentaires. Les contraintes essentielles sont aussi portées par PostgreSQL (quantité positive, acheteur différent du vendeur, valeurs de statut autorisées, unicité de la réservation), afin de rester garanties même en cas d'erreur applicative.

6.10.2 Gestion des pannes

Panne	Comportement prévu
Marketplace indisponible à la création	Erreur `503`, rien n'est enregistré.
Simulateur de paiement en erreur technique	Erreur `503` ; la transaction reste `PENDING` et le paiement peut être retenté. L'expiration protège la réservation.
RabbitMQ indisponible	Le changement de statut est validé ; l'événement reste dans `outbox_events` et est publié au retour du broker.
PostgreSQL indisponible	Erreur `503`, aucune modification ; les messages reçus ne sont pas acquittés et seront rejoués.
Message impossible à traiter	Trois tentatives, puis file de messages morts.

6.10.3 Concurrence et doubles opérations

— **Deux créations simultanées** pour la même réservation : la contrainte d'unicité n'en laisse passer qu'une, l'autre reçoit `409`.

— **Double paiement** : impossible grâce à la vérification du statut et à l'index unique partiel sur les paiements réussis.

— **Paiement et annulation simultanés** : le verrou optimiste détecte le conflit et la seconde opération reçoit `409`.

6.10.4 Cohérence entre services

Aucune transaction distribuée n'est utilisée : la cohérence repose sur la saga chorégraphiée décrite plus haut. Dans le service, chaque changement de statut, son historique, son paiement éventuel et son événement dans l'outbox sont écrits de manière **atomique**. La cohérence finale entre les services s'appuie sur l'idempotence des consommateurs.

6.10.5 Expiration des transactions

Une transaction non payée ne doit pas bloquer indéfiniment une quantité d'énergie. Une tâche planifiée annule les transactions `PENDING` dépassant le délai de 15 minutes (paramétrable) et publie `TransactionCancelled` pour libérer la réservation.

6.10.6 Sécurité des données

L'accès est protégé par l'authentification **Keycloak** et par la vérification, à chaque requête, que l'utilisateur agit pour l'acheteur ou le vendeur de la transaction. Les montants sont calculés par le service à partir des données du Marketplace.

6.11 Conclusion

Le microservice **Energy Transaction Service** formalise le dernier maillon du parcours d'échange d'EnergyShare : à partir d'une réservation du Marketplace, il calcule le montant, simule le paiement, suit le cycle de vie de la transaction, conserve son historique et délivre un justificatif.

Grâce à **Spring Boot**, **Spring Data JPA**, **PostgreSQL** et **RabbitMQ**, il combine une base transactionnelle fiable, une communication asynchrone sûre (outbox, idempotence, reprise) et des règles métier explicites. Son indépendance vis-à-vis des autres services, hors Marketplace, préserve l'autonomie de l'architecture en microservices.

Les contrats et mécanismes décrits constituent la cible de développement ; leur harmonisation avec le Marketplace et leur test restent à réaliser lors des étapes suivantes.

Chapitre 7

Microservice 5 – EnergyShare RAG Assistant

7.1 Objectif et périmètre

Le microservice **EnergyShare RAG Assistant** apporte une assistance conversationnelle spécialisée dans la documentation de la plateforme et les bonnes pratiques énergétiques. Il utilise la *Retrieval-Augmented Generation* (RAG) : avant de répondre, le système cherche des passages documentaires pertinents et les fournit à un modèle de langage. Il ne calcule pas les surplus, n'exécute pas de transactions et ne prédit pas automatiquement la consommation future.

7.2 Fonctionnalités principales

1. **Assistant conversationnel :** expliquer l'énergie solaire, le partage énergétique et les fonctionnalités d'EnergyShare.
2. **Recherche documentaire :** retrouver des informations dans des FAQ, guides solaires et documents du projet, puis restituer des réponses avec références.
3. **Conseils énergétiques :** proposer des bonnes pratiques de réduction de consommation et d'autoconsommation, dans les limites des sources accessibles.
4. **Contexte conversationnel :** conserver l'historique utile des échanges pour gérer les questions de suivi.

7.3 Stack technique

Composant	Technologie et rôle
Frontend	React + CSS, interface de chat.
Backend IA	Python + Flask, endpoints REST et orchestration du flux de requêtes.
Orchestration	LangChain, chargement documentaire, recherche et construction de prompts.
Chargement et découpage	TextLoader et RecursiveCharacterTextSplitter (passages documentaires).
Embeddings	Hugging Face, modèle `all-MiniLM-L6-v2` présenté comme option retenue.
Base vectorielle	ChromaDB, index des embeddings et métadonnées des passages.
Stockage relationnel	MySQL, conversations, messages et références d'utilisateurs Keycloak.
LLM	Ollama + Llama 3, génération de réponses (selon l'environnement de démonstration).

7.4 Architecture du pipeline RAG

Le microservice **EnergyShare RAG Assistant** repose sur une architecture de type **Retrieval-Augmented Generation (RAG)**, permettant de combiner une base documentaire spécialisée avec un modèle de langage afin de fournir des réponses contextualisées et accompagnées de sources.

Le pipeline commence par le chargement des documents de référence, tels que la documentation EnergyShare, les guides sur l'énergie solaire et les questions fréquentes. Ces documents sont ensuite découpés en fragments textuels (*chunks*), transformés en vecteurs sémantiques à l'aide de **Hugging Face Embeddings**, puis stockés dans **ChromaDB**.

Lorsqu'un utilisateur soumet une question, le système effectue une recherche sémantique dans la base vectorielle, construit un prompt en tenant compte du contexte conversationnel, puis interroge un **LLM** afin de générer une réponse pertinente.

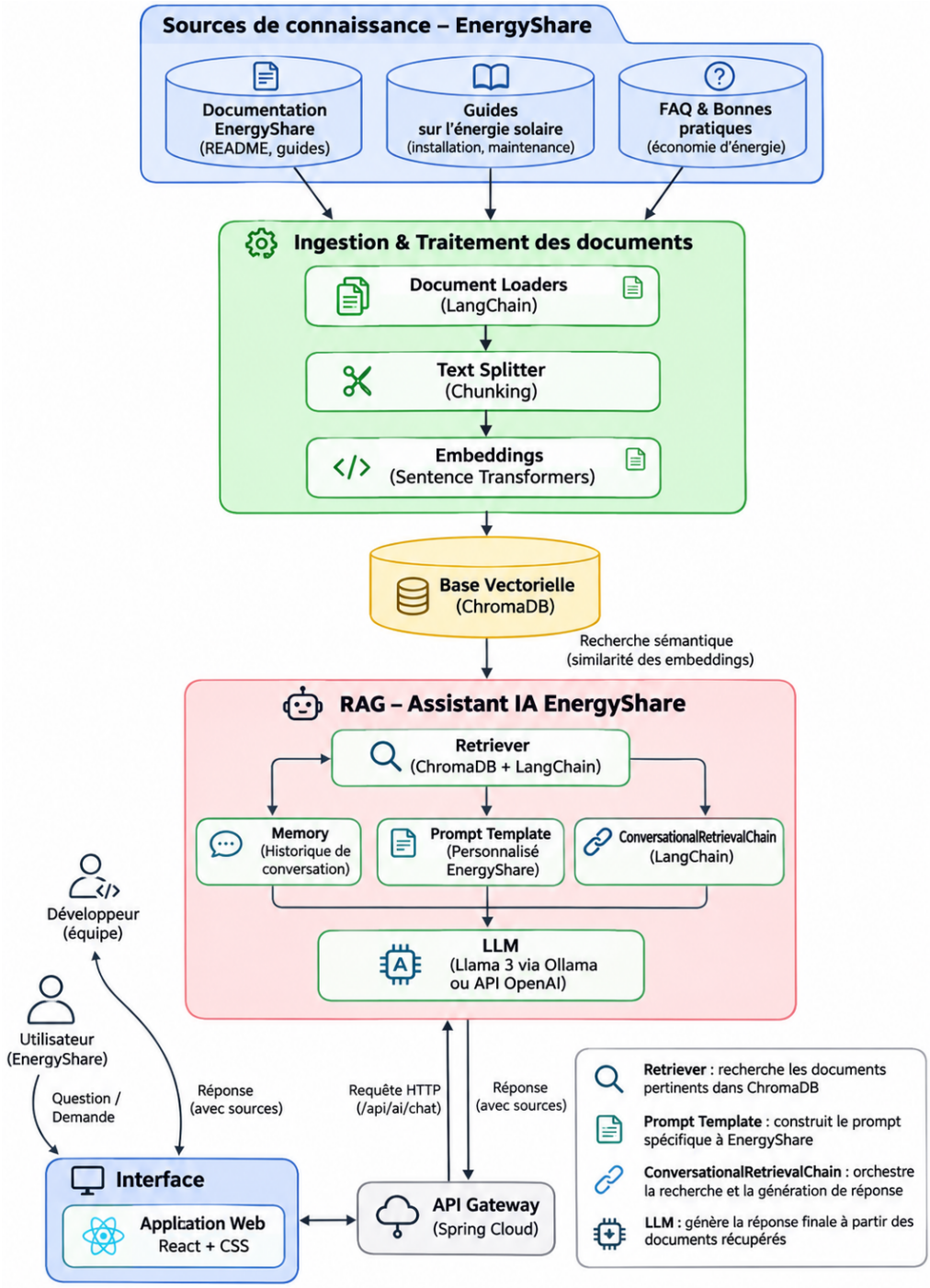


FIGURE 7.1 – Pipeline documentaire et de génération de réponse RAG (vue simplifiée).

La figure 7.1 illustre les principales étapes du pipeline RAG, depuis la préparation de la base de connaissances jusqu'à la génération de la réponse transmise au frontend via l'API Flask et l'API Gateway.

Cette architecture permet de structurer le fonctionnement interne du microservice 5 autour des composants suivants :

— **Sources documentaires** : documentation EnergyShare, guides solaires et FAQ ;

— **Prétraitement des documents** : chargement et découpage en fragments textuels ;

— **Vectorisation sémantique** : transformation des fragments en embeddings ;

— **Base vectorielle ChromaDB** : stockage des chunks et de leurs vecteurs ;

— **Retriever LangChain** : recherche des passages les plus pertinents ;

— **Construction du prompt** : combinaison du contexte retrouvé et de l'historique conversationnel ;

— **LLM** : génération de la réponse finale avec les sources ;

— **API REST Flask** : exposition du service au frontend React via l'API Gateway.

7.5 Préparation de la base de connaissances

Les fichiers de référence sont chargés puis divisés en passages (*chunks*) à l'aide de `RecursiveCharacterTextSplitter`. Chaque passage est transformé en vecteur avec le modèle d'embeddings Hugging Face et enregistré dans ChromaDB avec des métadonnées (titre, origine, section, identifiant). Cette préparation se fait lors de l'ingestion documentaire et n'a pas besoin d'être répétée à chaque question. Les documents doivent être sélectionnés, actualisés et vérifiés par l'équipe.

7.6 Cycle d'une question utilisateur

1. L'utilisateur écrit une question dans l'interface React.
2. L'API Gateway transmet la requête au service Flask.
3. Le service retrouve l'historique autorisé de la conversation dans MySQL.
4. La question est vectorisée ; le retriever récupère les passages pertinents de ChromaDB.
5. Un prompt structuré assemble question, contexte retrouvé et historique utile.
6. Le LLM génère une réponse ; le service restitue les références documentaires et enregistre l'échange.

7.7 Modèle relationnel et base vectorielle

Dans **MySQL**, on prévoit notamment `Conversation` (id, ownerUserId, title, createdAt) et `Message` (id, conversationId, role, content, createdAt). L'identifiant `ownerUserId` référence le compte externe de Keycloak ; MySQL ne réimplémente pas l'authentification. Les éventuelles informations de parcours de navigation pourront être ajoutées si elles sont utiles et consenties.

Dans **ChromaDB**, chaque enregistrement contient un identifiant, un vecteur d'embedding, le texte du passage et des métadonnées de source. La recherche sémantique ne garantit pas, à elle seule, la véracité des documents récupérés.

7.8 API REST prévisionnelles

Méthode	URI	Fonction	Succès
POST	`/api/ai/chat`	Envoyer une question et obtenir une réponse avec références	200
GET	`/api/ai/conversations`	Lister ses conversations	200
GET	`/api/ai/conversations/{id}`	Récupérer l'historique autorisé	200
DELETE	`/api/ai/conversations/{id}`	Supprimer une conversation	204
POST	`/api/ai/documents`	Importer un document (administration)	201

Les codes d'erreur envisagés incluent `400`, `401`, `403`, `404`, `429` (limitation de requêtes) et `503` (LLM ou moteur de recherche indisponible). Ces endpoints sont des **propositions de conception**. Le routage Gateway vers Flask utilisera une URL configurable dans l'environnement Docker.

7.9 Sécurité, fiabilité et limites

— Contrôler les jetons et l'accès à l'historique des conversations ; ne jamais exposer les conversations d'un autre utilisateur.

— Ne pas stocker de mot de passe utilisateur dans MySQL ; privilégier un identifiant externe lié à Keycloak.

— Renvoyer les sources récupérées pour permettre à l'utilisateur de vérifier les conseils.

— Prévoir un message de repli lorsque les documents ne permettent pas de répondre ou si le LLM est indisponible.

— Reconnaître les limites de génération : hallucinations, biais documentaires, sources dépassées et conseils non personnalisés.

— Encadrer l'ingestion : droits d'administration, validation et traçabilité des documents.

7.10 Évaluation de l'assistant

La validation qualitative peut mesurer la pertinence des passages retrouvés, la fidélité de la réponse aux documents, la qualité des références, le temps de réponse et la continuité des échanges. Un jeu de questions/réponses préparé par l'équipe permettra d'effectuer des tests répétables avant la démonstration.

Chapitre 8

Organisation du projet et perspectives

8.1 Arborescence du dépôt

```
EnergyShare/
|-- Backend/
|   |-- household-service/
|   |-- energy-monitoring-service/
|   |-- energy-marketplace-service/
|   |-- transaction-service/
|   `-- rag-assistant-service/
|-- Frontend/
|   `-- energyshare-react/
|-- Documentation/
|   |-- Presentation/
|   `-- Diagrams/
`-- README.md
```

Les services techniques (API Gateway, Eureka et Config Server), la configuration Keycloak et les fichiers Docker seront placés dans des dossiers adaptés à l'organisation Git de l'équipe.

8.2 Risques principaux

— **Disponibilité interservices :** établir des délais d'attente, des réponses d'erreur et des reprises.

— **Cohérence des réservations :** éviter les surréservations et traiter les doublons d'événements.

— **Qualité des données saisies :** valider les valeurs et préciser qu'elles sont déclaratives.

— **Qualité du RAG :** contrôler les documents, citations et réponses incertaines.

— **Intégration de Flask :** prévoir une route stable dans le Gateway et une stratégie d'authentification adaptée.

8.3 Conclusion

EnergyShare vise à démontrer comment une architecture microservices peut structurer une application de suivi et de valorisation des surplus solaires. La séparation entre Monitoring, Marketplace et Transaction rend les responsabilités métier explicites, tandis que le microservice RAG complète l'expérience utilisateur par une assistance documentaire spécialisée. La Validation 1 porte sur une **conception cohérente et des projets initialisés** ; les fonctionnalités, mécanismes de

sécurité avancés et intégrations décrits ici constituent la cible de développement et devront être testés lors des étapes suivantes.